



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

MASTER'S THESIS

**A Donor-Frame Coverage Metric for Evaluating
Occluded-Vehicle Completion in Automotive LiDAR**

Student:

Ngo Vi Viet Anh	MSE13205
-----------------	----------

Advisor:

Dr. Doan Nhat Quang

Master's Thesis

September 7, 2026

Keywords

- Autonomous driving perception
- Automotive LiDAR
- Point cloud segmentation
- Vehicle detection
- Point cloud completion
- Occluded-vehicle completion
- Donor-frame coverage metric
- Reference-free evaluation
- Amodal bounding box
- SemanticKITTI

Acknowledgements and Declaration of Authorship

Acknowledgements

I would like to express my sincere gratitude to my thesis supervisor, **Dr. Doan Nhat Quang**, for his guidance, constructive feedback, and support throughout the development of this research. His academic advice and technical insights have been essential to the conceptualization and refinement of the proposed framework.

I am also grateful to the faculty and staff at the **FPT School of Business & Technology (FSB)** for providing a structured academic environment and for the knowledge they have shared throughout the Master’s programme. I also thank my colleagues and peers for their helpful discussions and feedback during the research process.

Finally, I wish to thank my family for their patience and support during the period of my graduate studies.

Declaration of Authorship

I, **Ngo Vi Viet Anh**, student ID **MSE13205**, hereby declare that this thesis, entitled “*A Donor-Frame Coverage Metric for Evaluating Occluded-Vehicle Completion in Automotive LiDAR*”, is the result of my own work carried out under the supervision of Dr. Doan Nhat Quang at the FPT School of Business & Technology.

I solemnly declare that:

- The research presented in this work, including the donor-frame coverage metric, the hybrid detection-and-completion pipeline, and the evaluation of occluded-vehicle completion, is my own original work unless otherwise referenced.
- All experimental results and quantitative evaluations reported in Chapter 4 are authentic and have been obtained through testing on the specified datasets. No data fabrication or manipulation has taken place.
- All sources of information utilized in this thesis have been cited in accordance with standard academic referencing practices.
- This thesis has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

I take full responsibility for the content, accuracy, and academic integrity of this thesis.

Contents

1	Introduction	1
1.1	Background and Rationale	1
1.2	Challenges in Real-Data Completion	1
1.3	Research Objectives	2
1.4	Research Contributions	2
1.5	Thesis Organization	3
2	Literature Review & Theoretical Background	5
2.1	Literature Review	5
2.1.1	LiDAR Point Clouds and the SemanticKITTI Benchmark	5
2.1.2	Deep Segmentation of LiDAR Point Clouds	6
2.1.3	Classical Geometric Segmentation	7
2.1.4	Object-Level Point-Cloud Completion	7
2.1.5	Positioning of This Work	9
2.2	Theoretical Background	11
2.2.1	Ground-Plane Removal by RANSAC	11
2.2.2	Voxel-Grid Downsampling	12
2.2.3	Density-Based Clustering: DBSCAN and HDBSCAN	12
2.2.4	Permutation-Invariant Point Classification	12
2.2.5	Point-Set Distances: Chamfer, IoU, and Donor Coverage	13
3	Proposed Methodology	14
3.1	System Architecture	14
3.2	Detection Pipeline	14
3.2.1	Overview of the Modular Detection Pipeline	16
3.2.2	Preprocessing: Filtering, Downsampling, and Denoising	17
3.2.3	Ground-Plane Removal by RANSAC	17
3.2.4	Object-Candidate Clustering with HDBSCAN	18
3.2.5	Geometric Filtering of Non-Car Clusters	18
3.2.6	The Car / Not-Car Point Classifier	19
3.2.7	Multi-Frame Tracking and Track-Level Filtering	20
3.2.8	The Frozen Pipeline Configuration	21
3.3	Object Completion	22
3.3.1	Completion Targets: Single-Frame Car Clusters	23
3.3.2	The Completion Network (PCN)	23
3.3.3	Training Data: KITTI-Like Partial	24
3.3.4	Inference-Time Partial Normalisation	24
3.3.5	L-Shape Gating of Completion Inputs	26

3.3.6	Per-Car Length Estimation	26
3.3.7	The End-to-End Completion Path	27
3.3.8	Negative Results and Lessons	27
3.4	The Donor-Frame Completion Metric	29
3.4.1	The Invalidity of Chamfer Distance on Real Cars	29
3.4.2	The Donor-Frame Occluded-Side Principle	30
3.4.3	Definition of the Donor-Frame Coverage Metric	30
3.4.4	Summary and Methodological Limitations	33
4	Experimental Evaluation	34
4.1	Dataset	34
4.2	Experimental Setup	35
4.2.1	Baseline Methods	36
4.2.2	Evaluation Metrics	36
4.2.3	Implementation Details	37
4.3	Detection and Classification Results	37
4.3.1	The Reported Operating Point on Sequence 08	37
4.3.2	Per-Frame Detection Results	38
4.3.3	Leave-One-Sequence-Out Cross-Validation	38
4.3.4	Detection Accuracy Versus Range	41
4.4	Runtime Evaluation	42
4.5	Completion Results	43
4.5.1	Validation of the Donor Metric	43
4.5.2	Real-Data Surface Coverage	45
4.5.3	Downstream Utility: The Amodal Bounding Box	47
4.5.4	Moving Cars: A Plausibility Check	48
4.5.5	A Pre-Registered Held-Out Replication	50
4.6	Ablation Studies	52
4.6.1	Detection Ablations	52
4.6.2	Completion Ablations	54
4.7	Result Analysis and Error Analysis	57
4.7.1	The Shortfall as a Clustering Failure	57
4.7.2	Root Cause: HDBSCAN Fragmentation of Large, Close Cars	58
4.7.3	Limits of the Repair Strategies	59
4.7.4	Clustering Resolution and the Corrected Ceiling	62
4.7.5	Recall Analysis: Summary	63
5	Conclusion and Future Work	64
5.1	Conclusion	64
5.2	Limitations and Discussion	65
5.2.1	Established Results and Their Reach	65
5.2.2	Validity of the Evaluation	66
5.2.3	Scope of the Completion Claims	66
5.2.4	Ground-Truth and Dataset Assumptions	67
5.2.5	Architectural and Operational Limits	67
5.3	Future Work	67

List of Figures

3.1	Detection pipeline	16
3.2	Clustering stage	18
3.3	Dual-branch classifier	19
3.4	Tracker and track-level filter	21
3.5	Completion network (PCN)	23
3.6	Training-input distribution	25
3.7	Completions on real seq-08 cars	28
3.8	Donor-frame occluded-side principle	31
4.1	Bird’s-eye view of detections on seq 08	39
4.2	Per-frame detection behaviour on seq 08	40
4.3	Donor metric on six seq-08 cars	46
4.4	Amodal box overlays on six seq-08 cars	49
4.5	Completions of moving-car tracks on seq 08	51
4.6	Canonical-frame vs. naive normalisation	55
4.7	Detection failure close-ups on seq 08	60

List of Tables

2.1	Completion metrics and recovered occluded surface	9
2.2	Positioning against representative SemanticKITTI methods	10
3.1	Frozen pipeline configuration	22
4.1	IoU-threshold sensitivity on seq 08	37
4.2	Reported detection operating point	38
4.3	Leave-one-sequence-out cross-validation	40
4.4	Detection stratified by range-to-ego on seq 08	42
4.5	Per-frame runtime by stage on seq 08	43
4.6	Four donor-metric validation checks	44
4.7	Pooled guard blind to a band-localised violation	44
4.8	Real-data surface coverage on the donor metric	45
4.9	Reference-free metrics vs. the donor metric	47
4.10	Amodal box quality: raw partial vs. completion	48
4.11	Completion plausibility by track motion	50
4.12	Held-out replication on seq 00	52
4.13	Stage ablation on seq 08	52
4.14	Cross-domain classifier transfer	54
4.15	Clustering-method benchmark	54
4.16	Length geometry on the amodal box	56
4.17	Ground-truth cars across HDBSCAN clusters	58
4.18	Recall-repair strategies	61
4.19	Effect of coarsening the clustering voxel	62

Chapter 1

Introduction

1.1 Background and Rationale

Autonomous driving depends on a perception system that can recover, several times a second, the three-dimensional layout of the scene around the vehicle: where the other road users are, how large they are, and which way they face. LiDAR is central to this task. It measures range directly, returning an unordered cloud of over a hundred thousand points per scan, largely invariant to lighting and carrying the metric depth that motion planning needs. The benchmarks that opened the field were built around it: the KITTI suite [1] paired a scanning LiDAR with calibrated cameras, and its point-wise successor SemanticKITTI [2] turned the same sequences into the dense segmentation benchmark much of the field is now measured against.

The dominant paradigm is to train a deep network end to end on annotated scans: point-based segmentation networks [3, 4, 5], projection- and voxel-based models [6, 7, 8], single-pass object detectors [9, 10], and panoptic models [11, 12]. These define the state of the art in accuracy, which is paid for in two costs. The first is supervision: the results rest on dense, point-wise labels, and the annotation behind a benchmark like SemanticKITTI — a class on every one of billions of points — is out of reach for most groups and most new datasets. The second is opacity: an end-to-end network spreads each decision across millions of weights, so when it misses a car there is no single stage to inspect or repair.

This thesis takes the opposite starting point, asking how far a *modular*, largely annotation-free pipeline can go: classical geometry for the parts geometry solves well, and small learned models only where discrimination needs them — an approach to LiDAR vehicle detection that has drawn renewed interest [13]. Clustering proposes candidate objects, a small binary *car/not-car* classifier keeps the vehicles, and each detected car is completed by a PCN network; every stage is separately inspectable, and the learned components are trained without point-wise labels. Chapter 3 gives the full pipeline (Figure 3.1).

1.2 Challenges in Real-Data Completion

It is the second half of this pipeline that raises the problem the thesis is built around. Completing a partially observed car — recovering the far side the sensor cannot reach — is attractive for downstream box estimation, and networks such as PCN [14] are well studied on synthetic benchmarks. On real LiDAR they cannot be evaluated: there is no clean ground-truth complete shape, and the obvious substitute — accumulating the same static car across many frames — is itself one-sided and, as this thesis shows, invalid as a

reference, because the raw partial input scores best against it. Before completion can be claimed to help, it must be possible to measure at all. This is where the topic narrowed: an initial aim of applying learned completion to real LiDAR ran into the fact that its quality could not be measured validly at all, and that gap — not the completion network itself — became the question the thesis is built around.

1.3 Research Objectives

The work is organised around two research questions about measuring and validating completion on real LiDAR; the central contribution answers them, and two further findings — reported below in their own right — record what the investigation revealed about the detection front-end along the way.

1. **A Valid Evaluation Method for Real-LiDAR Completion.** Establish how the quality of point-cloud completion can be evaluated on *real* LiDAR, where no clean reference shape exists and an accumulated pseudo-ground-truth is itself one-sided — a measurement question prior to any claim that completion helps (RQ1; answered in Section 4.5.1).
2. **Evidence That Completion Recovers Unseen Surface.** Determine whether learned completion recovers genuinely *unseen* surface, as opposed to redistributing the points it was already given; whether it improves downstream amodal bounding boxes on real LiDAR cars; and whether any such benefit survives on a held-out sequence under a pre-registered test (RQ2; answered in Section 4.5.1, with the held-out outcome reported exactly as it fell).

1.4 Research Contributions

The thesis makes a single scientific contribution, methodological rather than architectural, together with the engineering contributions that make it possible.

- **A Validated Metric for Occluded-Vehicle Completion on Real LiDAR.** Chamfer distance against an accumulated pseudo-ground-truth is invalid on real cars, so the thesis defines a donor-frame occluded-side coverage metric: it scores how much of the surface seen only in other frames of the same static car a completion recovers, with a hallucination guard against surface invented outside the car’s box, and validates it against a four-item gate. Applied to real sequence-08 cars, it shows that PCN recovers occluded surface several times above the symmetry-mirror baseline and improves amodal boxes on static, gate-passed cars, and that the benefit partially holds on a pre-registered held-out sequence (Section 4.5.1). The detection and completion stages are established building blocks, reimplemented and tuned here; the contribution is the measurement built on top of them.
- **A Modular, Near-Annotation-Free Detection-and-Completion Pipeline.** A pipeline in which classical geometry does the work geometry solves well and small learned models are used only where discrimination needs them, with every stage separately inspectable and its per-stage runtime cost measured rather than estimated.

- **An Amodal Ground-Truth Box Builder.** A procedure that accumulates a static car’s frames into an amodal bounding-box reference, used to measure completion’s effect on downstream box estimation where no external amodal reference exists.
- **Reproducibility Infrastructure.** The frozen configuration, deterministic seeds, and evaluation scripts on which every reported number rests.

Along the way the investigation produced two diagnostic findings: that recall is limited by density clustering splitting single cars into fragments rather than by the classifier, with a chain of repair strategies that fail to generalise (Section 4.7); and that synthetic pretraining is redundant for the classifier once real training clusters are plentiful (Chapter 4).

The directions that did not work (architecture swaps, real-data completion fine-tuning, a sparse-fallback length model) are reported as negative findings rather than omitted.

The detector is car-only: the classifier is binary (car / not-car). **The tracker is a filter only:** it removes flicker detections and is not counted among the contributions. **Completion is trained without real labels:** the network sees only synthetic partial renders, never a labelled real car — what makes the pipeline near-annotation-free, and the source of the sim-to-real questions Chapter 4 takes up. **The system is offline:** it runs at about 623.5 ms per frame (1.6 fps), and real-time is out of scope (Section 4.4). **Detection is reported at one operating point but cross-validated:** sequence 08 (4071 scans) is where the ablations, recall root-cause, and completion analysis are decomposed, and a leave-one-sequence-out cross-validation (Section 4.3.3) bounds the model-selection exposure to about two precision points. Throughout, “recall” is reported against its car-only, at-least-ten-surviving-points denominator (Section 4.2).

Approach and method. The investigation is empirical and proceeds in four steps, each held to a single frozen configuration for reproducibility. First, we assemble the modular pipeline and train its learned components without point-wise labels: ground removal and density clustering propose candidate objects, a binary classifier keeps the cars, and PCN completes each one. Second, we fix a reproducible detection protocol — point-level IoU matching against SemanticKITTI-derived ground truth, scored by precision, recall, F_1 , and mean IoU — and characterise the detector on real KITTI-style sequences, with leave-one-sequence-out cross-validation guarding against model-selection bias. Third, because Chamfer distance against an accumulated pseudo-ground-truth is invalid on real cars, we define the donor-frame coverage metric and put it through a four-item validation gate. Fourth, we use the validated metric to measure whether completion recovers genuinely unseen surface and improves amodal boxes, with the outcome reported exactly as it fell on a pre-registered held-out sequence. The perspective throughout is that geometry should be left to classical methods and learning reserved for where discrimination needs it, and that a completion claim on real data is only as trustworthy as the measurement behind it.

1.5 Thesis Organization

The remainder of the thesis is organised into five chapters.

Chapter 1: Introduction presents the background and rationale, the challenges of evaluating completion on real LiDAR, the research objectives, the contributions, and the organisation of the thesis.

Chapter 2: Literature Review & Theoretical Background places the pipeline against the deep-learning segmentation and point-completion literature, carries the posi-

tioning table that defends the comparison, and fixes the notation and standard definitions for the operators the pipeline is built from.

Chapter 3: Proposed Methodology describes every stage of the detection pipeline and the classifier’s training, the point-cloud completion method, and — as the chapter’s destination — the definition of the donor-frame coverage metric that is this thesis’s contribution.

Chapter 4: Experimental Evaluation evaluates the pipeline end to end: it fixes the detection protocol and reports the seq-08 operating point, the ablations, the distance breakdown, and the runtime; root-causes the recall shortfall to clustering fragmentation and reports the chain of negative repair strategies, which is also the bridge to completion; and turns to its centre, putting the donor metric through its validation battery and using it to report the real-data coverage results, the amodal-box utility, the moving-car case, and the pre-registered held-out replication.

Chapter 5: Conclusion and Future Work answers the two research questions, names every weakness the work is subject to, and lays out what remains.

Chapter 2

Literature Review & Theoretical Background

This chapter places the thesis in its research context. It reviews the two bodies of work the pipeline draws on — LiDAR point-cloud segmentation and object-level point-cloud completion — and then positions the specific design choices of this work against them. The organising contrast runs throughout: the dominant approach to LiDAR segmentation is a single end-to-end network trained on dense per-point annotation, whereas this thesis builds an interpretable pipeline from training-free geometric clustering plus a lightweight learned classifier, and treats completion as a separate, downstream problem. Section 2.1.5 makes that contrast quantitative in a positioning table. The chapter is in two parts. Section 2.1 is the literature review — what exists in LiDAR segmentation and object completion, and where this work sits within it — and Section 2.2 is a theoretical-background section that fixes the notation and standard definitions for the operators the pipeline is built from. The pipeline-specific architectures, configurations, and design choices of each component are deferred to the method chapter (Section 3 for detection, Section 3.3 for completion).

Scope of the review. The field around automotive-LiDAR perception is far larger than any one chapter can survey, so the review is bounded by relevance to the thesis’s own design and evaluation. Included are the deep and classical LiDAR-segmentation paradigms the detection front-end is built from and positioned against (Sections 2.1.2 and 2.1.3), and object-level point-cloud completion together with its evaluation metrics — the setting of the thesis’s contribution (Section 2.1.4). Left deliberately outside are camera-LiDAR fusion and SLAM or map optimisation, both orthogonal to a single-scan, LiDAR-only object pipeline, and full 3D object-detection benchmarks scored by bounding-box average precision, which measure a different output than the point-level clusters this pipeline emits — an exclusion argued in full where it naturally arises (Section 2.1.5). The conceptual grounding and notation needed to read this related work, but no more, are gathered in Section 2.2.

2.1 Literature Review

2.1.1 LiDAR Point Clouds and the SemanticKITTI Benchmark

A rotating LiDAR sensor returns, for each scan, an unordered set of 3D points sampled from the surfaces around the vehicle. The points are sparse, irregularly spaced, and strongly range-dependent: nearby objects are dense, distant ones may be a handful of

returns, and every object is seen from a single viewpoint and is therefore partial. These properties — irregularity, sparsity, single-view partiality — shape every method discussed below and every difficulty this thesis reports.

The KITTI Vision Benchmark [1] established the standard automotive LiDAR setting (a Velodyne HDL-64E on a car, with synchronised camera and pose). Its segmentation-oriented successor, SemanticKITTI [2], provides dense per-point semantic labels for the 22 sequences of the KITTI odometry split (sequences 00–10 are released with labels; the labels for sequences 11–21 are withheld and scored only through the online test server), and a later extension adds temporally consistent instance identities for a LiDAR-based panoptic benchmark [15]. Three tasks are defined on it. *Semantic segmentation* assigns every point a class and is scored by the mean intersection-over-union across classes (mIoU). *Instance segmentation* separates individual objects within the “thing” classes (car, pedestrian, cyclist, and so on). *Panoptic segmentation* unifies the two and is scored by panoptic quality (PQ), which factorises into a segmentation-quality term (SQ, the mean IoU of matched segments) and a recognition-quality term (RQ, an F_1 -like matching score); PQ^{Th} restricts PQ to the “thing” classes, and $PQ^\dagger$ is a variant that scores “stuff” by IoU. These definitions matter for Section 2.1.5, because they are the metrics the literature reports and none of them is the point-level per-class F_1 this thesis uses.

The decisive property of SemanticKITTI for this work is its *annotation cost*. The labels are dense and per-point across billions of points, and the panoptic extension adds a manual instance-association pass over sequences. This is the resource the mainstream methods below consume and the one this thesis is built to avoid: its segmentation stage requires no labelled point clouds at all, and its classifier is trained on clusters mined and weakly labelled automatically from the pipeline’s own output (Chapter 3).

2.1.2 Deep Segmentation of LiDAR Point Clouds

The current benchmark leaders are deep networks trained end-to-end on the dense labels. They differ mainly in how they impose structure on the irregular point set. *Projection-based* methods rasterise the scan into a 2D range image and apply image convolutions: SqueezeSeg [6] and RangeNet++ [7] are representative, and are fast because they reuse mature 2D machinery, at the cost of discretisation artefacts at depth discontinuities. *Point-based* methods operate on the raw points through permutation-invariant operators: PointNet [3] introduced the shared-MLP-plus-symmetric-pooling design, PointNet++ [4] added hierarchical local grouping, and RandLA-Net [5] made the point-based route scale to full driving scans through efficient random sampling. *Voxel-based* methods discretise 3D space and run sparse convolutions; Cylinder3D [8] uses a cylindrical partition matched to the sensor’s polar geometry and reports 67.8 mIoU on the SemanticKITTI test set, among the higher single-scan results.

Panoptic methods extend semantic backbones with instance reasoning. Panoptic-PolarNet [11] predicts instances proposal-free in a polar bird’s-eye-view; DS-Net [12] clusters in a learned embedding via a dynamic shifting module; Panoster [16] learns instance ids end-to-end; and EfficientLPS [17] targets an efficient architecture for the joint task. The dataset authors’ own baselines pair a semantic network with a 3D detector to supply instances [15].

Two properties are common to this whole family and frame the thesis’s alternative. First, all of them require the dense per-point (and, for panoptic, per-instance) supervision described above; their accuracy is inseparable from that annotation budget. Second, they

are end-to-end networks whose decisions are not individually inspectable — a misclassified region has no stage-level explanation. PointNet and PointNet++ are relevant to this thesis for a narrower reason as well: they are the backbone family from which its car/not-car classifier is built (Chapter 3). The classifier, however, is a small binary network applied to already-segmented clusters, not a full-scene segmentation model, and it is trained without any hand-drawn labels.

2.1.3 Classical Geometric Segmentation

Before end-to-end learning dominated the benchmarks, LiDAR objects were extracted by geometric reasoning, and that route is the one this thesis takes for the segmentation stage. The standard recipe removes the ground plane — commonly by RANSAC plane fitting — and clusters the remaining points into objects. The clustering step is where the design choice lives. Fixed-radius methods such as Euclidean clustering and DBSCAN [18] group points within a single distance threshold ε ; this is fast and fully interpretable, but a single ε cannot serve both the dense near field and the sparse far field of a LiDAR scan. HDBSCAN [19], in the widely used implementation of [20], replaces the fixed radius with a hierarchy over density levels and extracts clusters that are stable across scales, which is better matched to range-varying point density.

This clustering-plus-classification recipe remains an active design point wherever interpretability and real-time efficiency are valued over benchmark accuracy. Recent work for autonomous racing, for instance, pairs bird’s-eye-view clustering with a lightweight machine-learning vehicle classifier to reach long-range detection without a deep dense-feature network [13]. The pipeline in this thesis is of the same family — ground removal, clustering, then a learned per-object classifier — applied to the SemanticKITTI setting.

The appeal of this route is that it needs no training data for segmentation and every stage is inspectable. Its weaknesses are equally concrete and are documented directly in this thesis: density-based clustering has no notion of objectness, so it fragments a single car into several clusters when internal density gaps appear, and this fragmentation — not classification error — is the dominant recall limiter (Section 4.7). The positioning below should be read with that trade in view: the interpretability and near-zero annotation of the classical route are bought at a measurable recall cost relative to the supervised networks.

2.1.4 Object-Level Point-Cloud Completion

The second phase of the pipeline reconstructs a denser shape from the sparse, single-view partial cloud of a detected car. Point-cloud completion learns this partial-to-complete mapping. PCN [14] established the coarse-to-fine design — a PointNet encoder producing a global shape code, decoded first to a coarse set and then folded to a dense output — and remains a strong, compact baseline. Later work improves detail through attention and progressive refinement: PoinTr [21] casts completion as set-to-set translation with a geometry-aware transformer, SnowflakeNet [22] generates points by hierarchical “snowflake” deconvolution, and SeedFormer [23] introduces patch-seed representations. These models are trained on synthetic CAD shapes, most commonly ShapeNet [24], with partial inputs produced by virtual rendering.

Two points from this literature bear directly on later chapters. First, models trained on clean synthetic partials face a synthetic-to-real domain gap when applied to noisy, sparse

LiDAR clusters; this thesis adopts PCN and confronts that gap head-on (Section 3.3), and its comparison of PCN against a transformer alternative found no decisive real-data advantage for the heavier model (Section 4.5.1). Second, and more consequentially, the standard completion metric — Chamfer distance to a reference shape — presupposes a complete ground-truth surface, which real accumulated LiDAR does not provide: the accumulation is itself one-sided. Section 4.5.1 shows this makes pseudo ground-truth Chamfer scoring invalid on real cars and develops an alternative metric in its place. This should be distinguished from *semantic scene completion*, which fills a whole voxelised scene from a single scan; the present work completes individual segmented objects, a different and narrower task.

Because real automotive LiDAR offers no complete target, a small toolkit of completion metrics has grown up around the KITTI cars. The established quantitative measures are the *unidirectional* Chamfer and Hausdorff distances (UCD and UHD), which score a completion by how closely it reproduces the points of its own partial input: Chen et al. [25] apply UHD directly to KITTI cars, and P2C [26] uses UCD together with a region-aware variant (RCD) that localises the comparison around observed skeleton points but still anchors it to seen geometry. PCN [14] instead scores its KITTI completions by *Fidelity* (how well the input is preserved in the output), *Minimal Matching Distance* (Chamfer distance to the nearest complete car in a synthetic reference set), and *Consistency* (agreement between the completions of one instance across consecutive frames). Each of these references something other than the real occluded surface — the observed input (UCD, UHD, RCD, Fidelity), a synthetic prior (MMD), or the model’s own outputs (Consistency) — so none directly measures how much genuinely unobserved surface a completion recovers. Indeed, because a completion that merely copies its input is scored best by an input-referenced measure, these metrics are used in the unpaired-completion literature alongside a separate plausibility or adversarial term rather than as a standalone ranking of recovered surface. Closest to the present work, Ren et al. [27] complete vehicles in real traffic scenes and build a pseudo ground truth by accumulating each tracked instance across consecutive frames, then score the completed cloud against that accumulation with Chamfer-based distances. That accumulation is scored *as a whole*, however — indeed one of its three distance terms measures input fidelity directly — so it mixes the near surface the input already observed with the little new surface the other views add, a completion is not penalised for merely reproducing its input, and no out-of-box hallucination guard flags surface invented outside the car. Section 4.5.1 adopts the same multi-frame idea but restricts scoring to the surface the input frame did not see, and adds an explicit hallucination guard, precisely so that reproducing the input earns nothing.

Table 2.1 summarises this landscape against the two properties the completion claim needs: whether a metric credits genuinely never-observed surface — and how directly — and whether it guards against hallucination. The existing measures range from penalising added surface, through crediting it only indirectly or mixed with already-seen surface, to not measuring it at all, and none guards hallucination.

Table 2.1: Established real-LiDAR completion metrics, by what each scores a completion against, whether it bounds surface invented outside the car (“guard”), and how it credits recovered never-observed surface (“unseen”): *no* = does not credit it (the input-fidelity measures in fact penalise it); *indirect* = credited only through a proxy such as resemblance to a synthetic car; *partial* = credited, but mixed with already-seen surface; *yes* = scored in isolation. Each measure is valid for its own purpose; none, however, isolates recovered unseen surface or guards hallucination, which is what the donor metric of Section 4.5.1 adds.

Metric	Scored against	Guard	Unseen
UCD [26]	observed input	no	no
UHD [25]	observed input	no	no
Fidelity [14]	observed input	no	no
MMD [14]	synthetic CAD prior	no	indirect
Consistency [14]	model’s own frames (cross-frame)	no	no
meanCD [27]	full multi-frame accumu- lation	no	partial
Donor (ours)	cov. donor-only observed surface	yes	yes

2.1.5 Positioning of This Work

The pipeline studied here is a car-only instance detector built from training-free clustering and a binary classifier, followed by object completion. It occupies a different point in the design space from the benchmark leaders of Section 2.1.2. Table 2.2 sets that position out along the axes on which it actually differs — approach, segmentation supervision, and output type — rather than on an accuracy score. A shared accuracy column is deliberately omitted: the published methods are scored by mIoU and panoptic quality across all classes, this work by a car-only point-level F_1 , and placing those figures in one column would invite a comparison the metrics do not support. The accuracy gap is real, and is stated plainly in the text below the table instead.

Table 2.2: Positioning against representative SemanticKITTI methods by approach, segmentation supervision, and output type — the axes on which this work actually differs. A reported-accuracy column is deliberately omitted: the published methods are scored by mIoU and panoptic quality across all classes and this work by a car-only point-level F_1 , so no single accuracy column would be comparable across rows. The accuracy gap is stated in the text.

Method	Approach	Supervision	Output
RangeNet++ [7]	Range-image CNN, semantic	Dense per-point	Per-point class
RandLA-Net [5]	Point-based, semantic	Dense per-point	Per-point class
Cylinder3D [8]	Voxel (cylindrical), semantic	Dense per-point	Per-point class
RangeNet++ & PointPillars [15]	Semantic net + detector, panoptic	Dense per-point + boxes	Per-point class + instance
Panoptic-PolarNet [11]	End-to-end panoptic	Dense per-point + instance	Per-point class + instance
DS-Net [12]	End-to-end panoptic	Dense per-point + instance	Per-point class + instance
EfficientLPS [17]	End-to-end panoptic	Dense per-point + instance	Per-point class + instance
This work	Geometric clustering + binary classifier (+ completion)	None for segmentation; auto-mined weak labels for the classifier	Point-level car clusters (+ completion)

Read as requirements, the table states the thesis’s position plainly. On their own benchmarks the supervised networks reach 52.2 to 67.8 mIoU (semantic) and 37.1 to 57.4 PQ (panoptic) over the official test sequences 11–21, scored across all classes; this work reports a car-only point-level F_1 of 0.808 (recall 0.730 against the $\geq$ -10-surviving-points denominator of Chapter 4, precision 0.905) on sequence 08. These are not the same measurement — none of the published metrics is this work’s point-level car F_1 , and the published rows are all-class test-set scores while this work’s is a car-only, single-sequence one — so the lower accuracy is a difference of paradigm, not a lower score on a shared axis, and this work does not contest the gap. What the pipeline offers instead is a different set of properties: it segments without any labelled point clouds, it runs on a single consumer GPU without a training stage for detection (its runtime is reported in Section 4.4), and every stage of its decision is inspectable, so its failure modes can be — and are — root-caused rather than left opaque (the recall shortfall is traced to clustering fragmentation, not classifier error, in Section 4.7). The accuracy gap is therefore a deliberate consequence of choosing interpretability and near-zero annotation over benchmark accuracy, not an unexplained deficiency.

A related question the table anticipates is why the comparison is drawn against segmentation methods rather than 3D object detection, the task the “detector” label most naturally invokes. Detection benchmarks score oriented 3D bounding boxes by average precision, as in PointPillars [9] and CenterPoint [10]. This pipeline, however, emits

point-level clusters — it must, because those clusters are the input to the completion stage (Section 3.3) — and never fits an oriented box; box AP would require a box-fitting stage the pipeline deliberately does not have. Point-level IoU against the ground-truth car points is therefore the metric that matches the actual output, and the choice of IoU threshold and recall denominator that make that measurement precise — together with its sensitivity to those choices — is examined directly in Section 4.2.

This framing also sets the boundary of the thesis’s contribution. Classical ground-removal-plus-clustering detection — including recent clustering-plus-classifier detectors for LiDAR vehicles [13] — a lightweight multi-object tracking stage used only to filter transient detections across frames (in the tradition of 3D multi-object trackers such as AB3DMOT [28], and not a contribution of this work), and PCN-style completion are established building blocks, reimplemented and tuned here; the contribution is methodological: a validated metric for completion quality on real LiDAR where reference-based Chamfer distance is invalid (Section 4.5.1). Two diagnostic findings sit alongside it: the root-cause analysis of the recall bottleneck together with the chain of negative repair results (Section 4.7), and the finding that synthetic pretraining is redundant for the classifier at scale (Chapter 4). The positioning table is the backdrop against which the contribution is argued: this is an interpretable, annotation-light system whose value is in what it explains and measures, not in topping a benchmark it was never designed to win.

2.2 Theoretical Background

This section fixes notation and states the standard definitions for the operators the pipeline is built from, in the order a scan passes through them: ground-plane fitting, voxel downsampling, density-based clustering, the point-set classifier, and the point-set distances used for completion and its evaluation. Each is a textbook construction cited to its source; the pipeline’s own architectures and parameter choices are given in the method chapters. A single LiDAR scan is written as a finite point set $P = \{\mathbf{x}_i \in \mathbb{R}^3\}_{i=1}^N$.

2.2.1 Ground-Plane Removal by RANSAC

A plane is a pair $(\mathbf{n}, d)$ with unit normal $\mathbf{n} \in \mathbb{R}^3$, $\|\mathbf{n}\| = 1$, and offset d ; a point lies on it when $\mathbf{n} \cdot \mathbf{x} + d = 0$, and its signed distance to the plane is

$$\delta(\mathbf{x}; \mathbf{n}, d) = \mathbf{n} \cdot \mathbf{x} + d. \quad (2.1)$$

RANSAC [29] fits the ground plane by repeatedly sampling three points, forming the plane through them, and scoring it by its inlier set — the points closer than a tolerance τ_g ,

$$\mathcal{I}(\mathbf{n}, d) = \left\{ \mathbf{x} \in P : |\delta(\mathbf{x}; \mathbf{n}, d)| \leq \tau_g \right\}. \quad (2.2)$$

The candidate with the largest inlier set is kept and its inliers removed as ground. To draw at least one all-inlier sample with probability p , given inlier fraction w and $s = 3$ points per sample, the required number of iterations is

$$T = \left\lceil \frac{\log(1 - p)}{\log(1 - w^s)} \right\rceil. \quad (2.3)$$

2.2.2 Voxel-Grid Downsampling

To reduce the range-dependent density bias before clustering, the non-ground points are downsampled on a regular grid of side v . Each point maps to an integer cell index

$$\text{vox}(\mathbf{x}) = \left\lfloor \frac{\mathbf{x} - \mathbf{x}_{\min}}{v} \right\rfloor \in \mathbb{Z}^3 \quad (2.4)$$

(the floor taken per coordinate), and every occupied cell c is replaced by the centroid of the points that fall in it,

$$\bar{\mathbf{x}}_c = \frac{1}{|P_c|} \sum_{\mathbf{x} \in P_c} \mathbf{x}. \quad (2.5)$$

2.2.3 Density-Based Clustering: DBSCAN and HDBSCAN

The non-ground points are grouped into objects by density. DBSCAN [18] uses a fixed radius ε and a minimum count m : with the ε -neighbourhood $\mathcal{N}_\varepsilon(\mathbf{p}) = \{\mathbf{q} \in P : \|\mathbf{p} - \mathbf{q}\| \leq \varepsilon\}$, a point is a *core point* when

$$|\mathcal{N}_\varepsilon(\mathbf{p})| \geq m, \quad (2.6)$$

and clusters are the maximal sets connected through core points. A single ε cannot serve both the dense near field and the sparse far field of a scan. HDBSCAN [19, 20] removes the fixed radius. With the *core distance* $d_{\text{core}}(\mathbf{p})$ set to the distance from $\mathbf{p}$ to its m -th nearest neighbour, it defines the *mutual reachability distance*

$$d_{\text{mreach}}(\mathbf{a}, \mathbf{b}) = \max(d_{\text{core}}(\mathbf{a}), d_{\text{core}}(\mathbf{b}), \|\mathbf{a} - \mathbf{b}\|), \quad (2.7)$$

builds a minimum spanning tree under it, and condenses the tree into a hierarchy indexed by $\lambda = 1/d_{\text{mreach}}$. A candidate cluster C born at $\lambda_{\min}(C)$, losing each point $\mathbf{p}$ at level $\lambda_{\mathbf{p}}$, has stability

$$S(C) = \sum_{\mathbf{p} \in C} (\lambda_{\mathbf{p}} - \lambda_{\min}(C)), \quad (2.8)$$

and the flat clustering is the set of disjoint clusters of maximum total stability. Stability-based extraction is what lets one setting cluster both near and far cars; it is also, as Section 4.7 shows, what splits a single car when an internal density gap makes two sub-clusters individually more stable than the whole.

2.2.4 Permutation-Invariant Point Classification

The car/not-car classifier is built on PointNet [3], whose defining property is invariance to the ordering of its input points. A function of a point set is made permutation-invariant by applying a shared map h to each point, pooling with a symmetric operator, and passing the pooled feature through a final map g :

$$f(\{\mathbf{x}_1, \dots, \mathbf{x}_n\}) = g\left(\max_{i=1, \dots, n} h(\mathbf{x}_i)\right), \quad (2.9)$$

the maximum taken coordinate-wise over the per-point feature vectors. Because the max pool is symmetric, f is independent of the order of the points; h and g are multilayer perceptrons learned end-to-end. Chapter 3 applies this design to a single cluster.

2.2.5 Point-Set Distances: Chamfer, IoU, and Donor Coverage

Completion and its evaluation are expressed through distances between point sets. The *Chamfer distance* between sets A and B averages, in each direction, the squared distance from every point to the nearest point of the other set,

$$\text{CD}(A, B) = \frac{1}{|A|} \sum_{\mathbf{a} \in A} \min_{\mathbf{b} \in B} \|\mathbf{a} - \mathbf{b}\|^2 + \frac{1}{|B|} \sum_{\mathbf{b} \in B} \min_{\mathbf{a} \in A} \|\mathbf{b} - \mathbf{a}\|^2, \quad (2.10)$$

and is the standard training objective for completion on synthetic shapes [30, 14]; Section 4.5.1 shows it is invalid as a real-data *score*. Detection is scored by a point-level intersection-over-union between a predicted cluster Q and a ground-truth car’s point set G ,

$$\text{IoU}(Q, G) = \frac{|Q \cap G|}{|Q \cup G|}, \quad (2.11)$$

a prediction matching a ground-truth object when $\text{IoU}(Q, G) \geq 0.3$ (Chapter 4). Completion is scored by the *donor coverage* developed in Section 4.5.1. Given the input cluster I , a *donor cloud* D of the same car’s labelled points accumulated from other frames, and a visibility radius τ , the *novel set* is the donor surface the input never observed,

$$\mathcal{V} = \left\{ \mathbf{d} \in D : \min_{\mathbf{p} \in I} \|\mathbf{d} - \mathbf{p}\| \geq \tau \right\}, \quad (2.12)$$

and the coverage of a candidate cloud C at radius r is the fraction of the novel set it reaches,

$$\text{cov}_r(C) = \frac{1}{|\mathcal{V}|} \left| \left\{ \mathbf{d} \in \mathcal{V} : \min_{\mathbf{c} \in C} \|\mathbf{d} - \mathbf{c}\| < r \right\} \right|. \quad (2.13)$$

By construction the raw partial covers none of $\mathcal{V}$; the full construction, its mirrored baseline, and its per-band hallucination guard are developed in Section 4.5.1.

Three points emerge from this review and set up the rest of the thesis. First, the accuracy of mainstream LiDAR segmentation is inseparable from a dense per-point annotation budget, which motivates the interpretable, near-annotation-free clustering-plus-classifier route taken here — and whose recall cost the later chapters measure and root-cause. Second, that route’s stage-level transparency is what makes such root-causing possible at all, where an end-to-end network offers no stage to inspect. Third, and most consequentially for the contribution, completion on real LiDAR has no valid reference-based score: every established metric is anchored to the observed input, a synthetic prior, or a one-sided accumulation, so none measures recovered unseen surface. These three points are the basis on which the thesis is built — the last is the gap its two research questions address (Chapter 1), by defining and validating a donor-frame coverage metric and using it to test whether completion adds genuine unseen surface and improves downstream boxes. The chapter has also fixed the notation and standard operators the later chapters build on. Chapter 3 turns those operators into the detection pipeline and the completion method, and develops the donor-frame coverage metric sketched here in full.

Chapter 3

Proposed Methodology

This chapter presents the proposed method: a modular detection-and-completion system for automotive LiDAR, and the donor-frame metric that makes its completion measurable on real data. Section 3.1 gives the overall architecture; the sections that follow describe each stage in the order a scan flows through it, and the completion is evaluated with the donor metric in Section 4.5.1.

3.1 System Architecture

The method is a single system that turns a raw LiDAR scan into completed, box-estimated cars together with a quality measurement for each completion. It is organised as three stages in sequence, each the subject of one section below. The *detection* stage (Section 3.2) segments and classifies car clusters from the raw scan, emitting an oriented bounding box and a stable track identity per car. The *completion* stage (Section 3.3) reconstructs each detected car’s full surface from the one-sided partial a single frame observes. The *measurement* stage — the donor-frame coverage metric (Section 3.4) — makes that completion quantifiable on real data, where no clean ground-truth complete shape exists. Figure 3.1 shows the detection chain that produces every input the later two stages consume, annotated with the measured per-stage cost.

The design is deliberately modular and classical: classical geometry does the work geometry solves well, and small learned models are used only where discrimination needs them, so every stage is separately inspectable and its cost separately measurable. This architecture is what lets the evaluation (Chapter 4) attribute the recall limit to a single stage rather than to an opaque network.

3.2 Detection Pipeline

This section describes the car-detection pipeline that produces every detection evaluated in Chapter 4 and every cluster completed in Section 3.3 and Section 4.5.1. The design is deliberately modular and classical: a chain of independent stages, each with a small number of interpretable parameters, rather than an end-to-end learned detector. This choice trades peak accuracy for interpretability, reproducibility, and near-zero manual annotation — the classifier is trained on labels mined automatically from the pipeline itself (Section 3.2.6) — and it is what lets Section 4.7 root-cause the recall shortfall to a single stage rather than to an opaque network.

The pipeline takes one raw LiDAR scan and emits a set of car detections, each an oriented bounding box with a stable track identity across frames. Every stage parameter lives in a single frozen configuration, fixed before the write-up began; the frozen values are collected in Table 3.1. Figure 3.1 gives the full chain with the measured per-stage cost.

3.2.1 Overview of the Modular Detection Pipeline

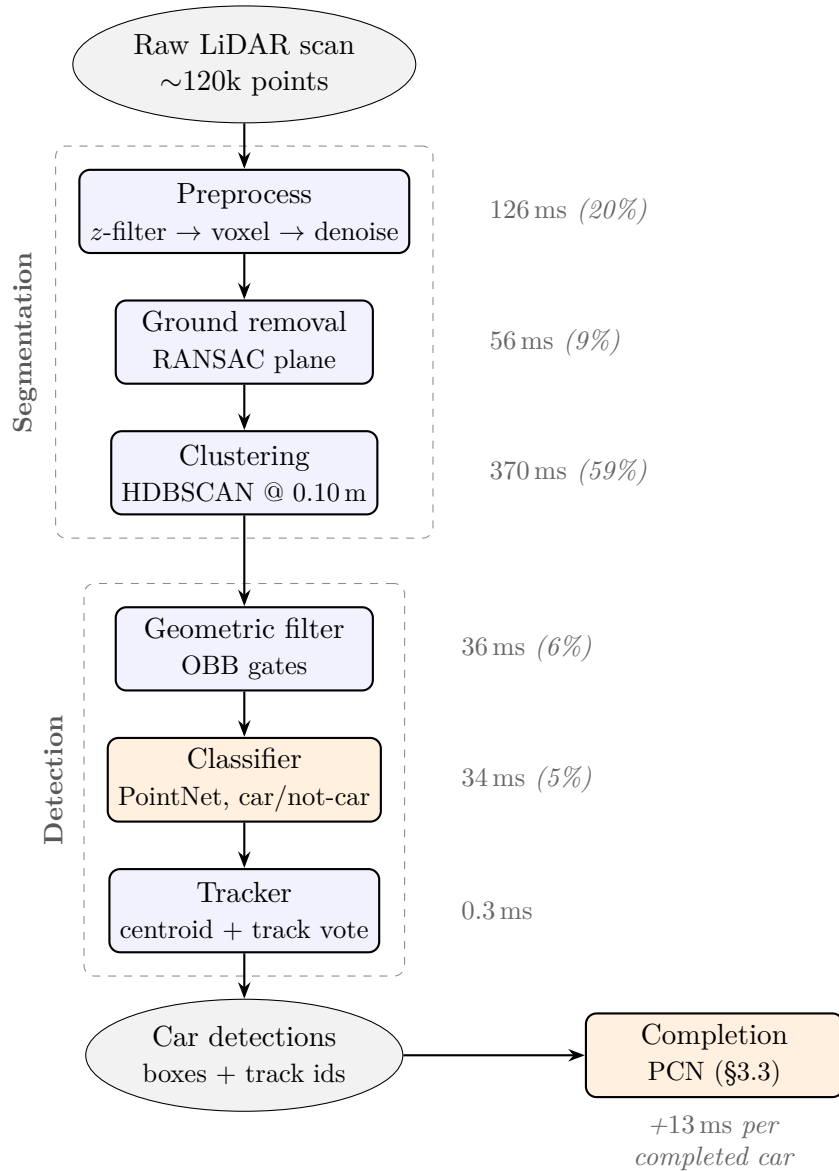


Figure 3.1: The detection pipeline, grouped into two phases (dashed): a *segmentation* phase that turns a raw scan into clusters (preprocess, ground removal, clustering) and a *detection* phase that turns clusters into tracked car boxes (geometric filter, classifier, tracker); the optional completion stage (right) runs off the tracked detections. Learned stages are shaded; all others are classical. Per-stage figures are mean wall-clock time on the full sequence 08 under the final (promoted) configuration (4068 frames, AMD Ryzen 7 7800X3D + RTX 3070 Ti), and are measured, not estimated. The total is $623.5 \text{ ms frame}^{-1}$ ($\approx 1.6 \text{ fps}$); the only learned *per-frame* stage — the classifier — accounts for just 34 ms (5%), and the classical clustering and ground-removal stages dominate. Completion runs off the tracked detections and is optional; it is learned, but its cost is per completed car, not per frame, so it does not enter this per-frame budget.

The system is *offline* by design: the tracker and the track-level filter use the whole sequence, and real-time operation is out of scope. At $623.5 \text{ ms frame}^{-1}$ the pipeline is roughly 1.6 fps, an order of magnitude below sensor rate; we characterise this cost honestly

in the runtime section of Chapter 4 rather than claiming real-time capability. The cost lives in the classical stages: clustering alone is 59 % of the frame budget, and the only learned per-frame stage, the classifier, is just 5 %. The expensive parts of this pipeline are classical, not neural.

3.2.2 Preprocessing: Filtering, Downsampling, and Denoising

Each raw scan (roughly 120 000 points) passes through three cleaning steps before any object reasoning:

1. **Ground-height z -filter.** Points below $z = -2.0$ m in the sensor frame are dropped. This removes the bulk of the road surface and any sub-ground noise cheaply, before the more expensive steps see them.
2. **Voxel downsampling** at 0.05 m. Points are collapsed to one representative per 5 cm voxel, which both regularises the range-dependent density of LiDAR and cuts the point count by roughly an order of magnitude.
3. **Statistical outlier removal.** Points whose mean distance to their 20 nearest neighbours exceeds 2.0 standard deviations of the local distribution are discarded, removing isolated speckle.

The order of the last two steps is behaviour-changing. In the frozen configuration (`voxel_before_denoise = True`) downsampling runs before denoising, so the statistical outlier test operates on roughly ten times fewer points — and on a regularised point set, which changes which points survive. This ordering was adopted as part of the pipeline optimisation primarily because it improves detection (discussed in Chapter 4); its effect on the measured denoising cost is small. It is behaviour-changing, so we flag it here rather than presenting the two orderings as equivalent. Preprocessing costs $126 \text{ ms frame}^{-1}$ in total, split across the z -filter (58 ms), denoising (50 ms), and downsampling (18 ms).

3.2.3 Ground-Plane Removal by RANSAC

The dominant remaining structure after preprocessing is the ground plane, which must be removed before clustering or it bridges every object into one component. We fit a single plane with RANSAC: over 300 iterations, three points are sampled, their plane computed, and inliers counted within 0.2 m; the best-supported plane is kept. The plane is *accepted only if its normal is near-vertical* — the z -component of the unit normal must exceed 0.5 — so a spuriously-fitted vertical surface (a wall, the side of a truck) cannot be mistaken for ground. If the check fails, no points are removed and the whole cloud is passed on as objects; this fail-safe is rare but prevents a bad plane fit from deleting real cars.

A single global plane is a deliberate simplification: it assumes the local ground is approximately planar, which holds on the flat urban sequences used here but would degrade on strong slopes. A grid-based ground model (Patchwork++-style) is listed as future work, not a limitation we claim to have solved. The iteration count was reduced from 1000 to 300 as part of the runtime optimisation after confirming the plane fit converges well before 1000 iterations on this data; the fit uses a fixed seed (`np.random.default_rng(42)`) so ground removal, and therefore the whole pipeline, is deterministic across runs. Ground removal costs 56 ms frame^{-1} (9 % of the frame).

3.2.4 Object-Candidate Clustering with HDBSCAN

Non-ground points are grouped into object candidates with HDBSCAN, a density-based hierarchical clusterer, using a minimum cluster size of 10 and minimum samples of 5. HDBSCAN is chosen over a fixed-radius method because LiDAR density falls off with range: a single Euclidean radius that merges nothing at 5 m will over-segment at 40 m. HDBSCAN’s variable-density hierarchy adapts to this without a hand-tuned radius schedule, and Chapter 4 reports a controlled benchmark confirming it beats DBSCAN and Euclidean extraction on recall at matched settings.

Two details of the frozen configuration are load-bearing. First, clustering runs on a coarser 0.10 m voxel grid than the 0.05 m preprocessing grid: the object cloud is re-downsampled to 0.10 m, HDBSCAN is run there, and the labels are propagated back to the full-resolution points by nearest neighbour. This was adopted because the coarser grid closes intra-car density gaps that otherwise cause a single car to split into several clusters — the central mechanism of the recall bottleneck analysed in Section 4.7 (Figure 3.2) — and it added 1655 true positives on the full sequence 08. Second, HDBSCAN is by far the most expensive stage at $370 \text{ ms frame}^{-1}$ (59 % of the budget), because it is superlinear in the point count; the coarse-grid trick is as much a speed measure as an accuracy one. The clustering stage produces on average 45 candidate clusters per frame on sequence 08, varying with scene density.

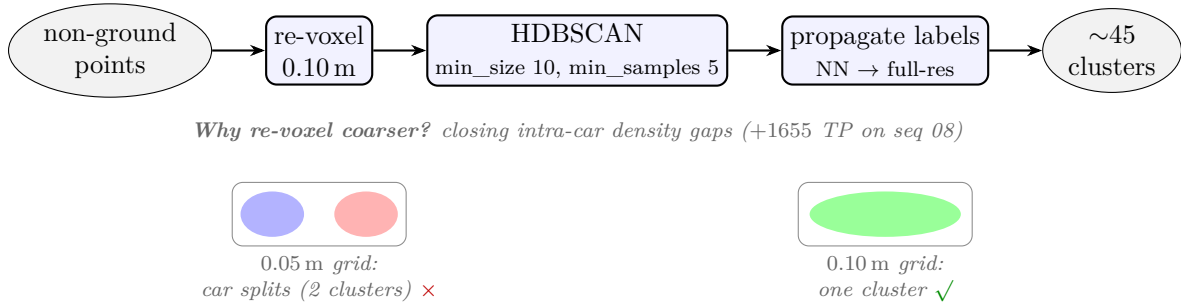


Figure 3.2: The clustering stage. Non-ground points are re-voxelised to a coarser 0.10 m grid, clustered with HDBSCAN (density-adaptive, so a single car stays whole at both near and far range without a hand-tuned radius), and the cluster labels are propagated back to the full-resolution points by nearest neighbour. The coarser grid is load-bearing (bottom): on the 0.05 m preprocessing grid intra-car density gaps split one car into several clusters, whereas the 0.10 m grid closes them and keeps the car a single cluster — worth 1655 true positives on sequence 08. HDBSCAN is the pipeline’s most expensive stage at $370 \text{ ms frame}^{-1}$ (59 %), so the coarse grid is also a speed measure.

3.2.5 Geometric Filtering of Non-Car Clusters

Most clusters are not cars: they are trees, poles, building facades, and fragments. A geometric filter rejects clusters whose oriented bounding box (OBB) is inconsistent with a car, using only cheap, interpretable shape gates. For each cluster the OBB is computed and the cluster is kept only if it passes all of the following:

- at least 15 points;
- OBB volume in $[0.5, 50] \text{ m}^3$;

- longest OBB extent ≤ 6.0 m, and the two largest extents above small floors (0.5 and 0.4 m), rejecting slivers;
- box centre no more than 1.5 m above the fitted ground plane, and vertical point span ≤ 1.8 m, rejecting tall structures;
- aspect ratio (longest / shortest extent) ≤ 6.0 , rejecting thin walls and poles.

These bounds are generous car priors, not a tight car model — they are meant to remove obvious non-cars while leaving the fine car/not-car decision to the learned classifier. The height gate is computed *relative to the fitted ground plane*, not to a fixed world height, so it is robust to the sensor not being perfectly level. The filter costs 36 ms frame^{-1} (6 %). Its individual gates are ablated in Chapter 4; the stage is high-recall and low-precision by construction, and the classifier is what turns its survivors into precise detections.

3.2.6 The Car / Not-Car Point Classifier

Each surviving cluster is classified as *car* or *not-car* by a small dual-branch PointNet. The binary decision replaces an earlier multi-class (car/bus/motorcycle) design that was dropped once the data showed near-zero real bus and motorcycle instances, leaving the car-only detector (Chapter 1).

Architecture. The network has two branches (Figure 3.3) and about 80 000 parameters — deliberately small:

- a **point branch** that takes 512 points (sampled or padded) normalised into the unit sphere, and applies shared per-point MLPs ($3 \rightarrow 64 \rightarrow 128 \rightarrow 256$) followed by a max-pool to a 256-d shape descriptor;
- a **bounding-box branch** that takes an 8-dimensional metric feature vector (sorted OBB extents, volume, two aspect ratios, $\log(1 + \text{point count})$, vertical span), z -score normalised, through a small MLP to 32-d.

The two are concatenated and passed through a small head ($288 \rightarrow 128 \rightarrow 2$) with dropout. The bounding-box branch is what re-injects absolute metric scale, which the unit-sphere normalisation deliberately strips from the point branch: a distant small car and a nearby one have the same normalised shape but different real extents, and the metric branch is how the network sees the difference. There are no T-Nets, since clusters already sit in a consistent sensor frame.

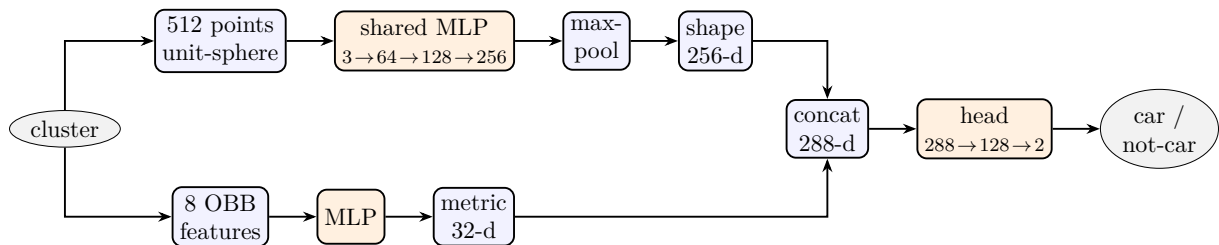


Figure 3.3: The dual-branch classifier applied to one cluster. The point branch takes 512 unit-sphere-normalised points to a 256-d shape descriptor; the bounding-box branch takes 8 metric shape features to 32-d. The two are concatenated and passed through a small head to a car/not-car decision. Learned blocks are shaded.

Training data (mined, not hand-labelled). The classifier is trained on real LiDAR clusters mined automatically from SemanticKITTI. For each training frame the pipeline’s own stages 1–5 are replayed, the ground-truth semantic labels are propagated onto the surviving clusters by nearest neighbour, and each cluster is labelled *car* or *not-car* by majority vote, keeping only clusters whose majority class has purity ≥ 0.75 . This yields 420 333 training clusters (sequences 00–07, 09, 10) and 130 394 validation clusters (sequence 08), with the intrinsic $\approx 21\%/79\%$ car/not-car imbalance handled by a class-weighted loss. The key property is that the training clusters are produced by the same clustering and filtering the classifier meets at inference — including HDBSCAN splits and filter survivors — so it learns on exactly the distribution it must classify, at no manual-annotation cost.

Production checkpoint: trained from scratch. The production classifier is `stage_b_scratch_best.pth`, trained on the mined real clusters from random initialisation (val macro- F_1 0.9285). An earlier two-stage design first pretrained on synthetic ShapeNet partial renders (“Stage A”) and then fine-tuned on the real clusters (“Stage B”). We report the two-stage machinery in Chapter 4 as an *ablation*, because a controlled comparison showed the synthetic pretraining to be redundant given 420 000 real clusters: the from-scratch classifier matches the pretrained one (macro- F_1 0.9285 vs. 0.9225). The final pipeline therefore uses the simpler from-scratch checkpoint, and the synthetic pretraining, together with the finding that the sim-to-real gap is total and symmetric, is presented as a negative result rather than as a contribution. The classifier costs 34 ms frame^{-1} (about 0.75 ms per cluster over the 45 clusters in a frame).

Training. The from-scratch checkpoint is trained for 15 epochs with Adam (learning rate 10^{-4} , weight decay 10^{-5}) and a step schedule that halves the rate every 10 epochs, at batch size 64. The loss is cross-entropy weighted by inverse square-root class frequency (normalised to unit mean), which offsets the intrinsic $\approx 21\%/79\%$ car/not-car imbalance without resampling. Training clusters are augmented online with a random rotation about the vertical axis and Gaussian jitter ($\sigma = 0.005 \text{ m}$); the input is the 512-point cluster of the architecture above. Runs are seeded and the reported checkpoint is the epoch with the best validation macro- F_1 (0.9285 on the sequence-08 validation clusters).

3.2.7 Multi-Frame Tracking and Track-Level Filtering

Detections are linked across frames by a centroid tracker. Each frame, current detections are matched to existing tracks by solving a Hungarian assignment on the pairwise centroid-distance matrix; matches beyond 2.0 m are rejected, unmatched detections start new tracks, and a track that goes unmatched for more than 5 consecutive frames is retired. This is an offline, centroid-only tracker — no motion model, no appearance — which is adequate here because detection, not association, is the bottleneck; an IoU-based (SORT-style) tracker is listed as future work.

The tracker also enables a *track-level filter* that improves precision (Figure 3.4). Each detection carries the classifier’s per-frame label; a track is accepted as a car only if, across its lifetime, it collected at least 2 non-rejected votes, those known votes are a majority of its votes (≥ 0.5), and they agree on a single class. A cluster that the classifier flickers on — a car in one frame, not-car in the next — is thus resolved by temporal majority rather

than trusted per frame. Because this filter consults the whole track, it is an offline post-processing step, consistent with the offline scope of the system; its contribution is isolated as an ablation row in Chapter 4. The tracker itself is effectively free at $0.3 \text{ ms frame}^{-1}$.

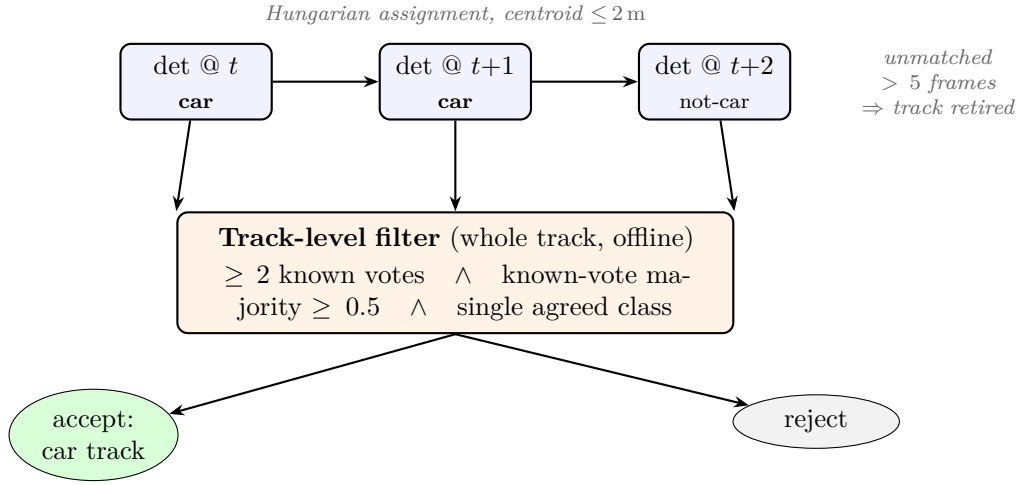


Figure 3.4: The tracker and track-level filter. Per-frame detections are linked into tracks by Hungarian assignment on centroid distance (matches beyond 2m rejected; a track unmatched for more than 5 frames retired). The track-level filter then resolves each track over its whole vote sequence rather than per frame: a track is accepted as a car only with at least 2 known votes, a known-vote majority of at least 0.5, and agreement on a single class. In the worked example the votes {car, car, not-car} clear all three gates, so a classifier flicker in one frame is overruled by temporal majority. Both steps are offline; the tracker costs $0.3 \text{ ms frame}^{-1}$.

3.2.8 The Frozen Pipeline Configuration

Table 3.1 lists the frozen parameter values that produce every number in this thesis. They are the committed PIPELINE_CONFIG and were not altered to obtain any reported result.

Table 3.1: Frozen pipeline configuration. These values were fixed before the write-up; no reported number was produced by changing them. The classifier is the committed from-scratch checkpoint (`stage_b_scratch_best.pth`).

Stage	Parameter	Value
Preprocess	z -filter threshold	-2.0 m
	voxel size	0.05 m
	denoise neighbours / std-ratio	$20 / 2.0$
	voxel before denoise	true
Ground	RANSAC iterations	300
	inlier distance	0.2 m
	min. normal z	0.5
Clustering	method	HDBSCAN
	min. cluster size / min. samples	$10 / 5$
	clustering voxel size	0.10 m
Geometric	min. points	15
	volume range	$[0.5, 50]$ m ³
	max. longest extent	6.0 m
	max. height above ground	1.5 m
	max. vertical span	1.8 m
	max. aspect ratio	6.0
Classifier	model	dual-branch PointNet (binary, from scratch)
	input points	512
Tracker	max. match distance	2.0 m
	max. disappeared frames	5
	min. track length	2
	min. known votes / ratio	$2 / 0.5$

The completion stage that operates on the tracked car detections (Figure 3.1, right) is the subject of Section 3.3; the evaluation of the detections themselves is Chapter 4.

3.3 Object Completion

This part of the method takes each detected car and completes it: from the one-sided, partial cluster a single LiDAR frame observes, it predicts the full car surface. The section presents the finished method in the order a cluster passes through it — the network, its training data, the inference-time normalisation, the input gate, and the length geometry — and then, separately, gathers the negative results and lessons that shaped it (Section 3.3.8). Those false starts are of independent value — one is the motivating case for a project-wide rule — so they are reported as findings rather than smoothed over, but they are kept out of the method description so that reads as what the pipeline does, not as how it was arrived at. The scientific content here is not the network, a standard Point Completion Network (PCN) used off the shelf, but the data it is trained on and the geometry by which a real partial is presented to it. The evaluation of the finished method — whether it adds genuinely unseen surface, and whether that generalises — is the subject of Section 4.5.1;

here we state each step’s headline effect and defer the rest.

3.3.1 Completion Targets: Single-Frame Car Clusters

Completion runs on the tracked car detections of Section 3.2. The input is a *single-frame* cluster in the sensor frame, not the accumulated multi-frame track. This is a deliberate choice with an evidence basis: accumulating a moving car over many frames produces a motion smear up to several metres long, which is neither a valid completion target nor matchable to a car shape. A single frame is one clean, if partial, observation. The value proposition — that completing this partial improves the car’s amodal box and recovers surface the frame never saw — is what Section 4.5.1 measures; this section is how the completion is produced.

3.3.2 The Completion Network (PCN)

We use PCN (Yuan et al., 2018), an encoder–decoder for point clouds (Figure 3.5). The encoder is a stacked PointNet: shared per-point MLPs lift each point to a local feature, a max-pool summarises the whole cloud into a global vector, that global vector is concatenated back onto every point, and a second MLP-plus-max-pool stage produces a single 1024-dimensional shape embedding. The decoder is coarse-to-fine: a fully-connected stage predicts 1024 coarse seed points, then a *grid-folding* stage attaches a small 2D grid patch to each seed and learns to fold it into local surface geometry, densifying the coarse skeleton into the final cloud. Training minimises a two-term Chamfer loss, $CD_{\text{coarse}} + 0.5 CD_{\text{fine}}$; the fine term is down-weighted so the coarse skeleton is learned first. At inference the partial is subsampled to 256 points before it enters the encoder.

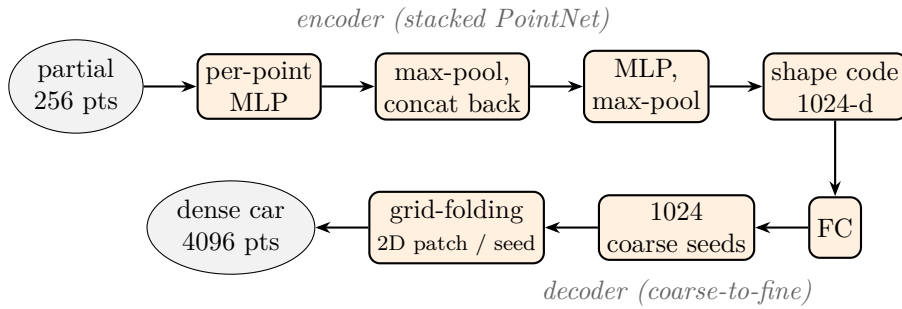


Figure 3.5: The completion network (PCN). A stacked-PointNet encoder maps the 256-point partial to a 1024-d global shape code; a coarse-to-fine decoder predicts 1024 seed points and folds a small 2D grid patch onto each to produce the dense 4096-point car. Every block is learned; the architecture is used off the shelf and unmodified.

The architecture is not a contribution and was not modified; the controlled comparison against PoinTr that justifies keeping PCN — a small synthetic edge and equivalent real-data behaviour — is reported with the completion ablations (Section 4.6.2). What makes PCN work on real cars is not the weights but the two subsections that follow: the distribution of partials it is trained on, and the frame a real partial is normalised into at inference.

3.3.3 Training Data: KITTI-Like Partial

PCN is trained on synthetic partials, and those partials must match the distribution of the real clusters it meets at inference or the network sees a different input at test time than it was trained on. A derisk study comparing candidate partial generators against real seq-08 clusters isolated three properties a naive generator gets wrong: it sampled a dense pinhole depth render, skipped the 0.05 m voxelisation and ground removal the real input has been through, and added range-proportional noise (up to 0.15 m at 30 m) that washed out the scan-ring banding real Velodyne data shows.

The generator used makes the synthetic partial match the real one: it orients the ShapeNet car to gravity, casts a single HDL-64E viewpoint (64 elevation beams, 0.09° horizontal resolution) from a sensor at 1.73 m and a sampled range of 8 m to 30 m, adds small *absolute* noise (0.015 m), voxelises at 0.05 m to match the pipeline, and removes points within 0.30 m of the ground plane. The dense complete mesh sample remains the target. PCN retrained on these partials (`pcn_kitti_best.pth`, best val loss 0.1246) is in-distribution with the real input, and on a held-out synthetic set produces clean cars, not blobs (Chamfer 0.16 m, F-score@0.1 m 0.76). Figure 3.6 contrasts the naive and KITTI-like partials for the same ShapeNet car.

Training. PCN is trained on the KITTI-like synthetic partials (input 256 points, target the dense mesh sample) for 100 epochs with Adam (learning rate 10^{-4} , weight decay 10^{-6}) and a step schedule that halves the rate every 40 epochs, at batch size 8, minimising the two-term Chamfer loss above. No real cars enter training; the reported checkpoint (`pcn_kitti_best.pth`) is the best-validation-loss epoch (0.1246). The subsequent geometry work is all inference-time and does not retrain the network.

3.3.4 Inference-Time Partial Normalisation

PCN is trained on shapes expressed in a fixed canonical car frame, centred on the true car centre and scaled by the true car radius, so at inference a real partial must be mapped into that same frame before it is completed — anything else hands the network a normalisation it never saw during training. Reconstructing that frame from a one-sided partial is non-trivial, because a partial reveals neither the true centre nor the true radius directly. The inference path (`estimate_canonical_frame`) reconstructs it in three steps:

- **Reorientation.** The sensor frame is rotated into the canonical car frame (X = width, Y = up, Z = length): gravity maps to the up axis, and the horizontal heading axis maps to length. Heading comes from the L-shape footprint fit (Section 3.3.5), not a principal-axis alignment.
- **Full-car centre estimate.** A one-sided partial’s bounding-box centre is not the car centre, so the centre is corrected on each axis: shift down to undo the upward bias from the ground cut (the dominant correction, 0.25 m); push outward along width toward the occluded far side to a half-car-width prior (1.9 m); and push along length toward the occluded far end (Section 3.3.6).
- **Scale.** Normalise by the maximum point-to-centre distance divided by a fixed scale correction (1.137), the calibrated ratio between a one-sided partial radius and the true car radius.

ShapeNet car 2668f2d9b8a2daad95551e0d9e50cb0d | naive dense/uniform/groundless vs KITTI-like single-view/voxelised/ground-cut

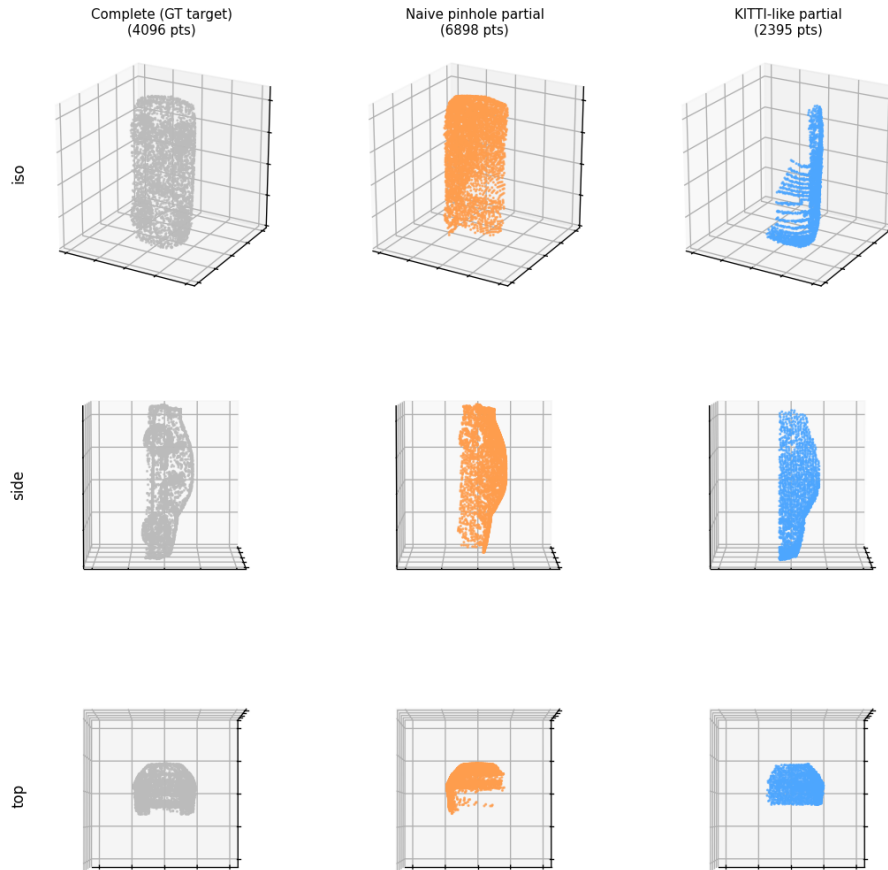


Figure 3.6: The training-input distribution: one ShapeNet car in three views, with its complete-surface target (left, grey) and two candidate training *inputs*. The naive pinhole partial (centre, orange) is dense and uniform; the KITTI-like partial (right, blue) reproduces what real LiDAR delivers — scan-ring banding, single-viewpoint coverage, 0.05 m voxelisation, and the ground cut. Neither is a completion output; “better” here means closer to the real input distribution, not more complete. Training on the KITTI-like distribution is what keeps the network in-distribution on real data.

With this path the same real seq-08 clusters recover the correct pose and scale; the ablation isolating this normalisation against a naive partial-frame alternative, and the box-overlap gains it produces, is reported with the completion ablations (Section 4.6.2). Viewed in the canonical car frame, the corrected completions are recognisably car-shaped: they recover the car footprint in bird’s-eye view and a rough side silhouette (roofline included) even from very sparse input — a clean car footprint from a 64-point cluster in the extreme case. We are nonetheless careful about the strength of the claim: crispness varies with input density (cleaner on well-observed cars, blobbier on the sparsest), and a completion is a filled 4096-point cloud rather than a CAD-clean surface. The sharpest completions are the *in-distribution synthetic* ones of Section 3.3.3 (Chamfer 0.16 m); on real data the normalisation’s value is corrected geometry — pose, scale, and the surface-coverage gains quantified in Section 4.5.1. A calibration study on synthetic cars further identified the residual dominant error as *centre estimation*, not scale — which is what motivates the geometry work in the rest of this section.

3.3.5 L-Shape Gating of Completion Inputs

Even with correct normalisation, some clusters cannot complete into a plausible car because the *input* is not one car: HDBSCAN fragments (part of a car) and merges (two cars, or a 90° heading error) both feed the completion network implausible input. To detect them we fit the cluster’s ground-plane footprint with a search-based L-shape fit (Zhang et al., 2017), which scores edge adherence over candidate yaw angles and stays well-defined on near-square footprints where PCA degenerates. The fitted footprint gives an input-quality gate: a cluster is rejected as a fragment if its fitted length is below 2.7 m, or as a merge if its fitted width exceeds 2.3 m.

On seq-08 every implausible completion originated from a fragment or merge input, so the gate’s effect on completion quality is a clean ablation, reported in Section 4.6.2. An honest secondary finding from the same study: the L-shape heading itself is neutral against plain PCA on real data — the fit earns its place by *detecting bad inputs*, not by giving a better heading.

3.3.6 Per-Car Length Estimation

The canonical-frame centre estimate re-centres the occluded lateral side and the ground cut, but a partial truncated at the occluded far *end* is normalised around its observed near portion, so PCN’s output stops short of the far end. The donor metric of Section 4.5.1 confirmed the far end as the single worst-covered region. Two successive fixes address it, both inference-only (no retraining).

The fixed length prior. A longitudinal, extend-only push mirrors the width prior along the length axis: the centre is pushed toward the occluded far end so the completion reaches an assumed full car length of 4.14 m (the real amodal-GT median). This more than doubled far-end coverage on the donor metric and reversed the box-length regression (signed ΔL $-0.44 \rightarrow -0.32$ m). A larger 4.5 m prior was rejected because it broke the hallucination guard by over-extending genuinely compact cars.

The per-car estimate. A single 4.14 m prior is a population median, so it is wrong in both tails — it over-extends compact cars and under-extends long ones. It is replaced by a

per-car length, `track_length_estimate`: the 90th percentile of the car’s own gate-passed footprint-length fits, plus a 0.12m bias correction, falling back to the fixed prior when a track has fewer than 5 gate-passed frames. The estimator was chosen offline against amodal GT with no PCN inference: a length-from-width shortcut is impossible (the correlation between true length and width is 0.018), and the aggregate must be a *quantile*, not the maximum, because a single contaminated L-shape fit sets the maximum. This is a per-track quantity, so the pipeline (which owns the track) computes it and passes it in; the completion module itself stays stateless. It removes the compact over-extension (compact signed ΔL 0.295→0.063 m) and improves the box metric overall (BEV IoU 0.747→0.771), at a measured cost in pooled donor coverage that Section 4.5.1 reports in full.

Limitation. The fallback is not rare: on the regenerated seq-08 output, 119 of 518 completed tracks (23%) fell back to the fixed prior because they had fewer than five gate-passed frames. This is above a comfortable threshold and is stated as a live limitation, not a solved problem; it is revisited in the discussion.

3.3.7 The End-to-End Completion Path

To summarise, a single-frame car cluster is completed by: fitting its footprint and rejecting fragments and merges (L-shape gate); reorienting it into the canonical car frame with no PCA; estimating the full-car centre (down-shift for the ground cut, width prior for the occluded side, per-car length estimate for the occluded far end); normalising by the scale-corrected radius; running the KITTI-like-trained PCN; and rotating the output back to the sensor frame. Every step is inference-time geometry except the one-off retraining on KITTI-like partials; no step fine-tunes on the evaluation cars.

Figure 3.7 shows what this path produces on real seq-08 cars across the input-density range, in the canonical car frame. The completed cloud (green) fills the car footprint and a rough side silhouette that the single-frame partial (blue) leaves one-sided — recognisably car-shaped down to a 64-point input, with fitted dimensions in the expected car range ($L \approx 4.2$ m, $W \approx 1.9$ m, $H \approx 1.6$ m on the well-observed cars). Consistent with the caveats above, the sparsest inputs complete more coarsely and can inflate the width (Section 4.5.1), and the clouds are filled rather than CAD-clean; the point here is qualitative shape, with the quantitative real-data assessment — whether the completion adds genuinely unseen surface, and whether that survives a pre-registered held-out replication — being what Section 4.5.1 sets out to measure.

3.3.8 Negative Results and Lessons

The method above was not reached in one step. Two false starts shaped it, and each carries a lesson worth stating in its own right rather than smoothing over. They are gathered here so that the method description reads as what the pipeline does. A third intervention that was tried and rejected — a length-dependent far-end compensation — is reported as a completion ablation (Section 4.6.2).

The blobs were not a synthetic-data problem. The first attempts to run PCN on real cars all produced “blobs” — rounded, car-sized clouds with no recognisable car structure. This was initially attributed to the training data being synthetic, and the response was a series of sim-to-real remedies: LiDAR-noise augmentation of the synthetic

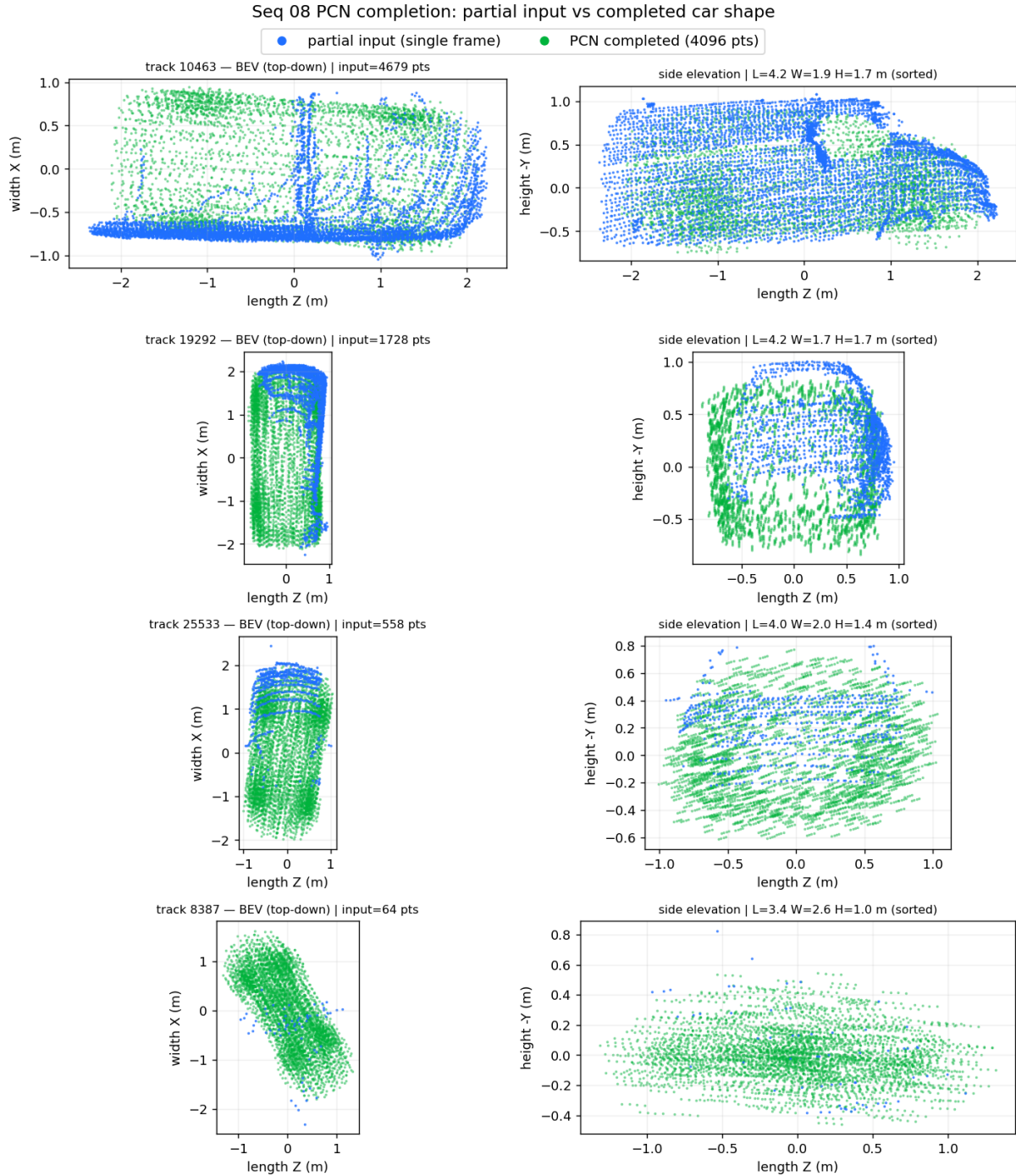


Figure 3.7: What the final completion path produces on real seq-08 cars, four tracks spanning the input-density range (4679 down to 64 input points), each in the canonical car frame: bird’s-eye view (left of each pair) and side elevation (right). Blue is the single-frame partial input, green the PCN completion. The completion recovers the full car footprint and a rough roofline silhouette from a one-sided partial; crispness and dimensional accuracy degrade on the sparsest inputs (the 64-point car shows the width inflation discussed in Section 4.5.1).

partials, and real-data fine-tuning. None of them fixed the blobs, and the fine-tuning attempt is recorded as a negative result — on this data, fine-tuning PCN on mined real pairs did not recover real-car geometry. The blobs had two causes, and neither was the network weights: the training-input distribution (Section 3.3.3) and the inference normalisation (Section 3.3.4), the two method steps above. The lesson is to suspect the data and the input pipeline before the model.

Train and inference semantics must match. Even after retraining on KITTI-like partials, real clusters still blobbed, and the cause is the sharper lesson. An earlier inference path normalised each real partial by a 3D PCA frame and the *partial's* own centroid and radius — a normalisation the model had never seen, since it was trained in the canonical car frame of Section 3.3.4. This silent train/inference mismatch alone produced blobs, and it did so for every checkpoint, including in-distribution synthetic ones (it worsened Chamfer by roughly $3.5\times$ even where the model is otherwise clean). Because the two frames each looked internally reasonable, the divergence was invisible until measured. This incident is the motivating example for the project's rule that any external integration or reuse gets a written semantics map — coordinate frame, centring, and scale conventions — reconciled before code is run.

3.4 The Donor-Frame Completion Metric

Evaluating the completion output requires a metric, and on real LiDAR there is no complete ground-truth surface to score against. This section explains why the obvious choice — Chamfer distance to a pseudo ground truth — is invalid, then defines the *donor-frame* metric used in its place. The metric is validated and applied to the completion output in Section 4.5.1; it is defined on static cars by construction, with moving cars treated separately in Section 4.5.4.

3.4.1 The Invalidity of Chamfer Distance on Real Cars

The natural way to score a completed car is Chamfer distance (CD) against a ground-truth surface. Real LiDAR offers no such surface: a car is seen once, from one side. The obvious substitute is a *pseudo*-ground-truth cloud — the labelled points of one static car accumulated across every ego frame that observed it. This subsection explains why that substitute is not a valid target, and hence why the completion work of Section 3.3 needed a different metric altogether.

The fundamental limitation is that the accumulation is itself one-sided. A parked car is observed from a road running past one of its flanks, so every frame sees broadly the same near faces — the road-facing side, the near end, the roofline edge — and none sees the occluded far side or the underbody. Pooling those frames densifies the visible surface but adds almost no genuinely new surface: the pseudo-ground-truth is a denser partial, not a whole car.

Scored against such a reference, plain Chamfer *rewards under-completion*. The distance is symmetric, so every predicted point pays the distance to its nearest reference point. A completion that correctly reconstructs the occluded far side places points where the reference has nothing, and each such point adds cost; a model that predicts only the already-observed surface pays nothing for what it omits. The raw partial — which by construction sits exactly on observed surface and predicts nothing new — therefore scores

the lowest (best) CD of any cloud, on every real example. A metric that ranks the un-completed input first cannot be used to measure completion.

This is a measured failure rather than a conjecture: across the real seq-08 cars we scored, the raw partial had the lowest pseudo-ground-truth CD of the raw / mirrored / completed clouds in every case. Its consequence is that when the geometry work began, completion had no valid real-data metric at all — the in-distribution synthetic CD of Section 3.3 (0.16m) measures the network on synthetic cars but says nothing about a real cluster. The next subsection turns the very one-sidedness that breaks Chamfer into the basis of a metric that works.

3.4.2 The Donor-Frame Occluded-Side Principle

The idea turns the one-sidedness of LiDAR from a problem into the measurement. For a static car observed across many frames, we complete it from one frame’s pipeline cluster and then score the completed cloud only against surface that the other frames — the *donor* frames — saw but the input frame did not (Figure 3.8). Two properties make this a fair test of completion:

- The raw partial covers none of that donor-only surface by construction, so any coverage the completion achieves is genuinely added, previously-unseen surface — not a rescoring of the input points.
- The reference is built from ground-truth labelled points only, never from a pipeline output, so no detection or completion error can leak into the thing we measure against. The metric is a paired comparison of raw versus completed on identical inputs, so it tolerates reference noise the same way the box evaluation does.

Relation to existing completion metrics. Completion on real LiDAR has been scored before without a complete target, but not against the previously-unseen surface specifically. The KITTI completion metrics introduced with PCN [14] — Fidelity, Minimal Matching Distance, and Consistency — reference the input, a synthetic car prior, and the model’s own cross-frame outputs respectively, so a completion that merely reproduces its input can score well on all three. Ren et al. [27] come closest to the construction used here: they too accumulate each tracked vehicle across frames to form a pseudo ground truth. The difference is what the accumulation is scored against. They compare the completed cloud to the accumulation in full, mixing the near surface the input already observed with the small amount the other views add; the donor metric keeps only the novel set — the surface at least τ from every input point — so already-observed surface contributes nothing and the raw partial scores zero by construction. This is also what separates the metric from semantic scene completion [2], which scores voxel occupancy over a whole scene rather than surface coverage of one object. The novel-set restriction, the mirrored baseline of Section 3.4.3, and the per-band hallucination guard of Section 4.5.1 are what make the metric measure occluded-surface recovery specifically, rather than overall resemblance to an accumulated cloud.

3.4.3 Definition of the Donor-Frame Coverage Metric

For each (frame, instance) true-positive pair on the well-observed static cars, we form three point sets and score them against a common target.

The donor principle: the novel set is occluded surface only other frames observed

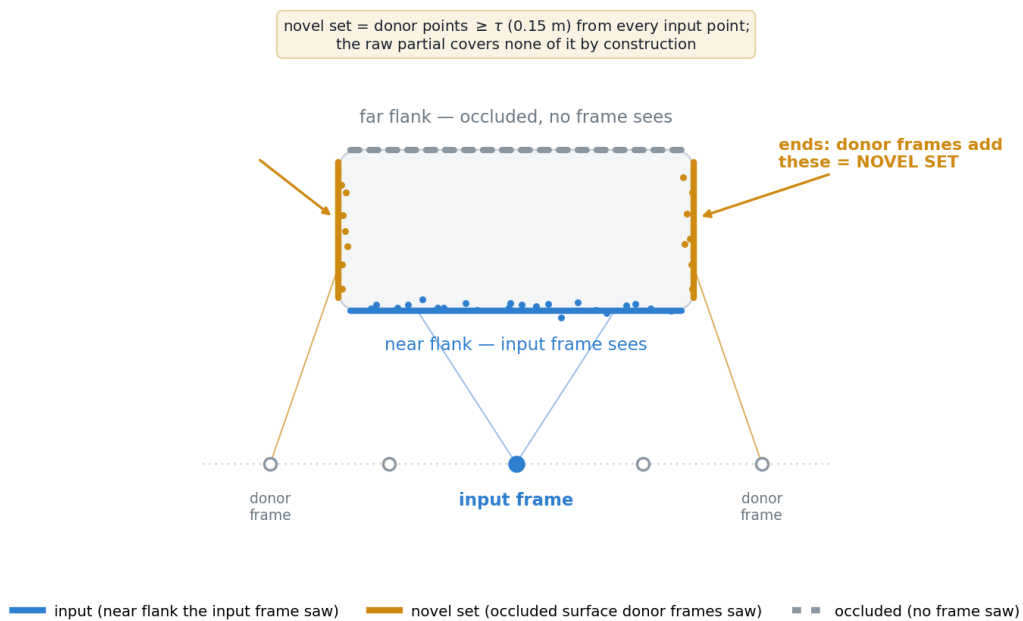


Figure 3.8: The donor-frame occluded-side principle (schematic; a car seen from above, the road below). The input frame observes the near flank (blue); as the ego drives past, the *donor* frames additionally observe the two ends (amber), which together form the *novel set* — the surface the metric scores against. The far flank (grey) is occluded from every frame and is not part of the reference. Because the novel set lies at least τ from every input point, the raw partial covers none of it by construction. Illustrative geometry, not measured data; real per-car coverage on seq 08 is Figure 4.3.

Inputs. The **input** is the production pipeline’s cluster for that car in that frame, completed through the production path (the KITTI-like PCN checkpoint behind the L-shape input gate). The **donor cloud** is the union of the instance’s ground-truth (labelled-car) points from every observing frame except the input frame, transformed to the world frame and voxelised at 3 cm.

Novel set (the visibility mask). From the donor cloud we keep only points that lie at least τ from every input point (nearest-neighbour test). This *novel set* is exactly the surface the input frame did not observe. We use $\tau = 0.15$ m as the primary radius and 0.10 / 0.20 m as stability checks.

Scored clouds. We score three clouds against the novel set so that completion is judged against meaningful baselines rather than in isolation:

- **raw** — the partial input (the floor: covers no novel surface by construction);
- **mirrored** — the input unioned with its own reflection across the car’s estimated symmetry plane (a cheap, hallucination-free way to add surface; the baseline completion must beat to justify a learned model);
- **completed** — the PCN output.

Scores. Each cloud gets three numbers, all one-directional from the novel set to the cloud:

- **coverage** (cov@0.1) — the fraction of novel points within 0.1 m of the cloud (the primary quality score: how much unseen surface was recovered);
- **median novel distance** — the median distance from novel points to the cloud (a threshold-free companion to coverage);
- **out-of-box** — the fraction of the cloud’s points falling outside the amodal ground-truth box (plus a 0.2 m margin): a *hallucination guard*, since a model can trivially raise coverage by inventing surface everywhere.

Eligibility. The metric applies to any completion whose input is a *single, well-formed object cluster* — one cluster corresponding to one car, with a footprint consistent with a whole vehicle — so that there is a coherent object shape to score. In this pipeline that admission is realised by the L-shape input gate of Section 3.3 (fitted footprint length ≥ 2.7 m and width ≤ 2.3 m, rejecting fragments and merges), but the metric itself does not depend on that particular gate: any admission rule that isolates a single object serves in its place.

Statistics. A pair counts if its input is admitted and its novel set has at least 100 points at the primary τ . Because the frames of one parked car are autocorrelated, we reduce each car to its per-car median and test across cars with the Wilcoxon signed-rank test — the same paired, per-car design used for the box evaluation.

3.4.4 Summary and Methodological Limitations

This chapter has set out the full method: the modular detection pipeline that turns a raw scan into car detections, the single-frame completion path with its input gate and length prior, and the definition of the donor-frame coverage metric. Several limitations are built into these choices and are flagged here so the evaluation is read with them in view; each is taken up in full in the discussion of Chapter 5.

- **Ground model.** A single global RANSAC plane assumes near-planar ground; it holds on the flat urban sequences used here but would degrade on strong slopes, where a grid-based model (future work) would be needed (Section 3.2.3).
- **Offline by design.** The tracker and track-level filter consult the whole sequence, and at ≈ 1.6 fps the pipeline is far below sensor rate; real-time operation is out of scope (Section 3.2.1).
- **Single-frame, static-car completion.** Completion runs on one frame’s cluster, and the donor metric is defined on static cars; moving cars are treated separately and are not scored for completion accuracy (Section 3.3.1).
- **Length-prior fallback.** The per-car length estimate falls back to the fixed 4.14 m prior on 23 % of tracks (those with fewer than five gate-passed frames), and a residual far-end under-extension on long cars is characterised, not eliminated, limit (Sections 3.3.6 and 4.6.2).
- **Synthetic-only completion training.** PCN sees only synthetic KITTI-like partials; real-data fine-tuning was a negative result, so a sim-to-real residual remains (Section 3.3.8).
- **Metric admissibility.** The donor metric requires a single well-formed object cluster and multi-frame donor observations, so it does not score fragments, merges, or movers (Section 3.4.3).

With these caveats stated, Chapter 4 puts each component to the test — the detection operating point and its ablations, the recall root-cause, and the donor metric’s validation and completion results — and Chapter 5 returns to the limitations above alongside the evaluation-validity questions the results raise.

Chapter 4

Experimental Evaluation

This chapter evaluates the full pipeline end to end. It first fixes the experimental setup — dataset, the pipeline under test, metrics, baselines, and test environment (Section 4.2). It then reports detection and classification results together with runtime (Section 4.3), and the completion results that carry the thesis’s contribution — the donor-frame coverage metric and what it reveals about real-data completion (Section 4.5). Two further sections decompose these results: ablations that isolate each stage’s contribution (Section 4.6), and an analysis of where detection succeeds and fails (Section 4.7), which root-causes the recall limit to the same clustering step that governs input cleanliness for completion — the link between the chapter’s two halves.

4.1 Dataset

We evaluate on the eleven labelled SemanticKITTI odometry sequences (0 to 10), whose per-point semantic and instance labels serve as ground truth. The official test sequences 11 to 21 carry no public labels, so no ground-truth detection metric can be computed on them; this is a property of the dataset, not a choice, and it is why no labelled sequence can be reserved as a conventional held-out test set.

Classifier evaluation uses leave-one-sequence-out cross-validation. Rather than report detection on the sequence the classifier was tuned against, we evaluate the binary classifier under *leave-one-sequence-out* (LOSO) cross-validation over all eleven sequences. For each held-out sequence K a classifier is trained on the other ten sequences and then run on the full sequence K ; the held-out sequence contributes zero clusters to that fold’s training data and is never used to select the checkpoint. Checkpoint selection is removed entirely: every fold trains for a fixed 15-epoch budget and the last-epoch checkpoint is used (historically the best epoch was 13/15, so last $\approx$ best), so the held-out sequence is untouched during both training and model selection. In the per-fold table below, therefore, the classifier scoring sequence K has never seen sequence K during training or model selection. Because the Stage B clusters are stored with a sequence prefix in their filenames, the folds are a re-partition of the existing mined data and require no re-mining.

Two sequences carry additional, distinct roles.

- **Sequence 08 (reported operating point; 4071 frames).** The largest labelled sequence is the operating point at which the pipeline is reported in detail: the stage

ablations (Section 4.6), the recall root-cause analysis (Section 4.7), the IoU-threshold sensitivity below, and the completion evaluation (Section 4.5.1) are all computed on the sequence-08 output of the final classifier. Sequence 08 was never used to tune the classical stages. The final classifier was historically selected on sequence 08, so its sequence-08 numbers could in principle be optimistic; the LOSO fold-08 classifier (trained on 0 to 7, 9 to 10, last-epoch, never selected on 08) settles this empirically below.

- **Sequence 00 (completion held-out replication).** Sequence 00 is the held-out sequence for the *completion* replication (Section 4.5.1), where its being unseen by the completion model is what matters; the test is a paired comparison over its 45 well-observed static cars (Section 4.5.5), not a frame-count sweep. Its classical pipeline parameters (PIPELINE_CONFIG) were historically hand-tuned on it, so the seq-00 detection fold retains classical-stage familiarity even though its classifier is still properly held out; we isolate that residual in the aggregate below (dropping fold-00 moves pooled recall by 0.022).

Ground-truth targets and the eligibility rule. The pipeline is a car detector, so the evaluation counts only the two car-bearing SemanticKITTI classes as valid ground-truth objects: *car* (class 10) and *moving-car* (class 252); we refer to this target set as *supported-vehicles*. A ground-truth car *instance* is a maximal set of points sharing one instance id within these classes.

An instance is counted in the evaluation only if at least 10 of its points survive preprocessing — the z -filter, voxel downsampling, statistical denoising, and ground removal that every scan passes through before clustering. This is the recall denominator: *recall is computed against ground-truth car instances with ≥ 10 points surviving preprocessing*, not against every annotated car. The rule exists because an instance reduced to a handful of points by occlusion or range cannot be segmented by any clustering method, so including it would measure sensor sparsity rather than the pipeline.

This choice narrows the denominator, and honesty requires quantifying by how much. On a stride-20 sample of sequence 08 (204 of 4071 frames, drawn uniformly across the drive), of all annotated car instances (2332 pooled), 25.5% are excluded by the surviving-points rule; measured instead against instances that have ≥ 10 points in the raw scan (2033 pooled), 14.6% are excluded by preprocessing.¹ Because an excluded instance can never be matched, the sequence-08 operating-point recall of 0.730 against the eligible denominator corresponds, by inference, to a recall of approximately 0.54 against all annotated cars (the eligible-to-annotated survival ratio on this sample is $1737/2332 = 0.745$; the ratio is a stride-20 estimate, so this figure is inferred, not separately measured). We report the eligible-denominator recall as the headline and disclose the all-annotated figure alongside it.

4.2 Experimental Setup

Every quantitative claim about detection in this thesis is produced by a single evaluation script under one fixed protocol. Because the results, ablation, and recall-analysis sections all report numbers from this protocol, we state it in full here and refer back to it rather than

¹These are two distinct ≥ 10 -point rules: one thresholds raw label points (a distinct-car count), the other thresholds points *surviving preprocessing* (the eval denominator). They are not interchangeable.

restating it per table. The protocol was frozen before the write-up began; no configuration, checkpoint, or metric definition described below was altered to produce a reported number.

4.2.1 Baseline Methods

Detection is not compared head-to-head against the supervised networks of Section 2.1.5: those report different metrics under different protocols, and Table 2.2 sets out that non-comparison and the paradigm gap behind it. Within this pipeline the classical stages and the track filter are instead compared against one another by ablation (Section 4.6). Completion is judged against explicit baselines on the same inputs: the *raw partial* (the floor — it adds no unseen surface by construction), its *symmetry mirror* (a cheap, hallucination-free way to add surface that a learned model must beat), and, for architecture, *PoinTr*; the standard reference-free metrics UCD and UHD (Section 2.1.4) are reported alongside the donor metric in Section 4.5.2.

4.2.2 Evaluation Metrics

Detections and ground-truth instances are compared at the *point level*. For a detection mask D and an instance mask G over the surviving points, the overlap is the intersection-over-union

$$\text{IoU}(D, G) = \frac{|D \cap G|}{|D \cup G|}.$$

All detection–instance pairs with $\text{IoU} \geq \tau$ are collected, sorted by descending IoU, and assigned *greedily and one-to-one*: the highest-IoU pair is matched first, and once a detection or an instance is used it cannot be matched again. This yields, per frame:

- a **true positive** (TP) for each matched pair,
- a **false positive** (FP) for each unmatched detection,
- a **false negative** (FN) for each unmatched eligible instance.

Precision, recall, and F_1 are then computed from TP, FP, and FN *micro-averaged* over the sequence: counts are pooled across all frames and the ratios taken once, so denser frames contribute proportionally more. We also report the **mean IoU of matched detections** — the mean of the IoU values of the accepted TP pairs only. This is a quality measure of the matches that were made; it says nothing about missed cars and must not be read as an overall segmentation score.

Track-level filtering is enabled by default for the headline numbers: an offline centroid tracker links detections across frames, and only tracks meeting a minimum length and class-consistency requirement are accepted. This is a post-hoc, offline step (it uses the whole sequence), consistent with the offline scope of the system; its effect is isolated as an ablation row in the ablation section (Section 4.6).

Choice of IoU threshold and its sensitivity. All headline numbers use $\tau = 0.3$. A point-level threshold of 0.3 is permissive relative to the bounding-box 0.5 common in detection benchmarks, so it is fair to ask whether the results are an artifact of a lenient gate. They are not, and the reason is visible in Table 4.2: the mean IoU of matched detections is 0.912 on sequence 08, far above either threshold, so the matches are not marginal overlaps sitting just past τ .

To make this concrete we re-ran the full sequence 08 evaluation at $\tau \in \{0.25, 0.30, 0.50\}$ with everything else frozen (Table 4.1).

Table 4.1: IoU-threshold sensitivity on the full sequence 08 (frozen configuration). The $\tau = 0.30$ row is the headline. F_1 moves by at most 2.6 points across the tested range (0.811 at $\tau = 0.25$ to 0.785 at $\tau = 0.50$); mean IoU of matched detections rises at the stricter gate because it keeps only the better-overlapping matches.

τ	Precision	Recall	F_1	Mean IoU	TP / FP / FN
0.25	0.908	0.732	0.811	0.910	25 576 / 2578 / 9346
0.30	0.905	0.730	0.808	0.912	25 478 / 2676 / 9444
0.50	0.879	0.709	0.785	0.927	24 744 / 3410 / 10 178

F_1 falls only 2.3 points from the headline $\tau = 0.30$ to the strict $\tau = 0.50$ (0.808 to 0.785), and 2.6 points across the full tested range from the permissive $\tau = 0.25$ (0.811), so the reported quality is stable under the choice of threshold and does not depend on the lenient value. We had pre-registered that a threshold-sensitive result would be *reported as a finding rather than tuned away*; it was not sensitive.

4.2.3 Implementation Details

The system under evaluation is the frozen detection-and-completion pipeline of Chapter 3, run unmodified by a single script: ground removal, HDBSCAN clustering, the geometric filter, the binary classifier, and the centroid tracker with its track-level filter for detection; and, for each detected car, the KITTI-like-trained PCN behind the L-shape input gate for completion. Every number below uses the committed PIPELINE_CONFIG (Table 3.1); no stage, checkpoint, or parameter was changed to obtain a reported result.

All runs are deterministic: ground-plane RANSAC and the classifier use fixed seeds (Chapter 3), so a rerun reproduces every reported number. Timings are wall-clock on an AMD Ryzen 7 7800X3D with an RTX 3070 Ti, measured on the full sequence 08 under the promoted configuration (Section 4.4). The classifier and PCN training hyperparameters — optimiser, learning-rate schedule, epochs, and batch size — are given with each model in Chapter 3 and are not repeated here.

4.3 Detection and Classification Results

4.3.1 The Reported Operating Point on Sequence 08

Table 4.2 gives the reported operating point on the full sequence 08 under this protocol, produced by the final classifier (the one the completion pipeline actually runs with). This is the operating point the ablation and recall-analysis sections of this chapter decompose; the cross-validation that confirms it follows in Section 4.3.3.

Table 4.2: Reported detection operating point on the full sequence 08 (frozen protocol, final classifier), with its leave-one-sequence-out held-out counterpart — fold-08, a classifier that never saw sequence 08 (Section 4.3.3). The two agree on recall (the held-out row is marginally higher) and differ by about two precision points, so selecting the classifier on sequence 08 did not inflate the reported recall. Sequence 08 is the largest labelled sequence and was never used to tune the classical stages; mean IoU is over matched detections only.

Classifier (seq 08)	Precision	Recall	F_1	Mean IoU	TP / FP / FN
Reported (final)	0.905	0.730	0.808	0.912	25 478 / 2676 / 9444
Held-out (LOSO fold-08)	0.881	0.737	0.803	0.912	25 733 / 3477 / 9189

The pipeline is precision-saturated and recall-limited: false positives are flat at roughly one per frame, while recall is set by how many eligible cars the clustering stage recovers as single objects. The remaining sections of this chapter report the ablations that decompose this operating point, and Section 4.7 root-causes the recall shortfall.

4.3.2 Per-Frame Detection Results

The headline operating point is the reported row of Table 4.2 (Section 4.2): precision 0.905, recall 0.730, F_1 0.808, and a mean IoU of matched detections of 0.912, over 25 478 / 2676 / 9444 TP / FP / FN. Two properties of this operating point organise the rest of the chapter, and both are visible directly in the per-frame record rather than only in the pooled totals.

First, the pipeline is *precision-saturated*. Across the full sequence the false-positive count is low and stable — about 0.7 per frame (2676 over 4071 frames), independent of how many cars are in view. Figure 4.1 shows this spatially — detections track the true parked and moving cars along the street, with few spurious boxes — and Figure 4.2 shows it over time: the FP trace is low and stationary while the number of ground-truth cars per frame swings widely.

Second, the pipeline is *recall-limited*, and the limit is structural. The recall shortfall traces to a measured clustering failure: HDBSCAN splits large, close cars into fragments that no longer match a single instance, and the split rate is highest exactly where cars are densest and nearest (66 % at 0 m to 10 m). Section 4.3.4 exhibits the resulting near-range recall dip directly, and the time series (Figure 4.2) shows per-frame recall tending to fall in the denser frames while false positives stay flat. We root-cause the splitting mechanism in Section 4.7; here we establish only that recall, not precision, is the dominant error mode, and that it does not come from the classifier misfiring (false positives stay flat regardless).

4.3.3 Leave-One-Sequence-Out Cross-Validation

The operating point above is reported on sequence 08, which is also the sequence the final classifier was historically selected on. To show that this selection does not inflate the result, and to characterise the per-sequence recall spread that a single reporting sequence cannot, we ran the LOSO protocol of Section 4.1: eleven folds, each training a classifier on ten sequences and evaluating the full held-out eleventh, so every fold’s classifier is blind to the sequence it scores. Table 4.3 reports all eleven folds, the pooled micro-average (counts summed across folds, comparable to the single operating-point number), and the macro mean $\pm$ standard deviation across folds.

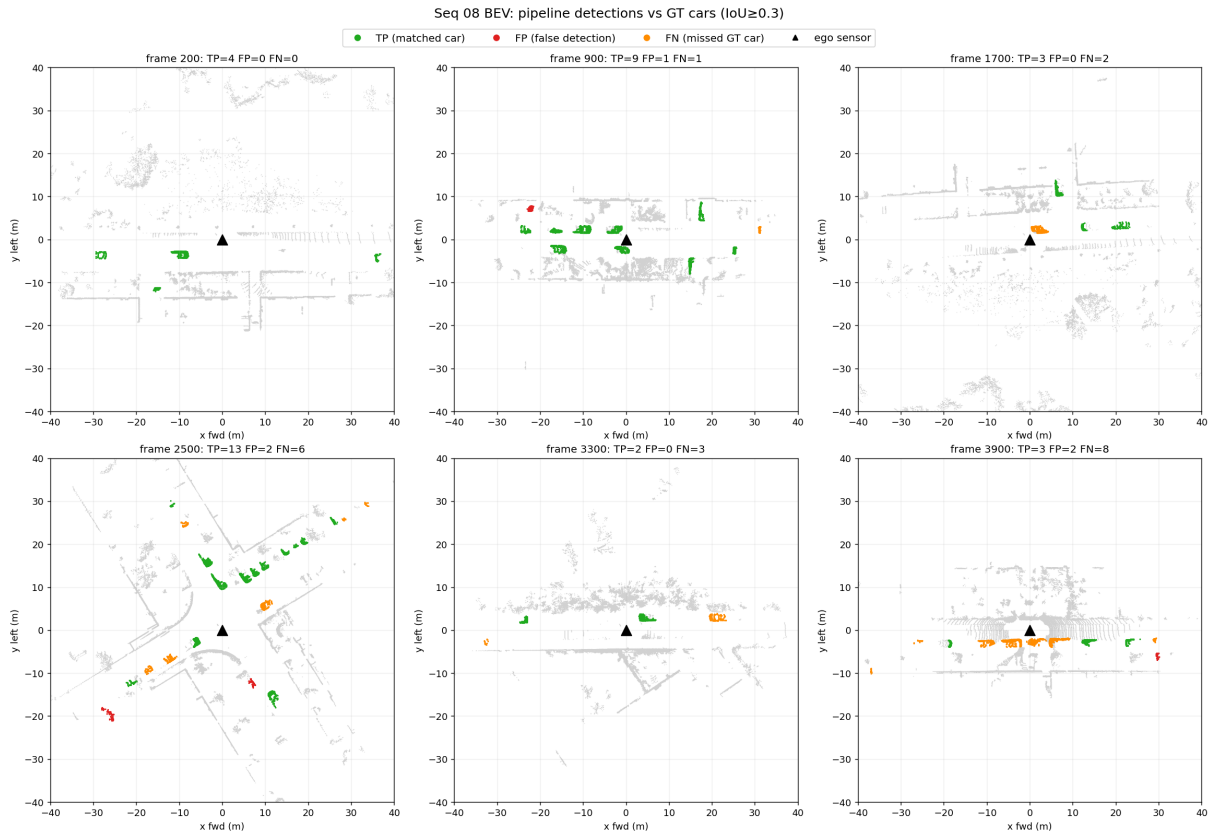


Figure 4.1: Bird's-eye view of detections on a representative sequence 08 frame. Detected car clusters (coloured) are overlaid on the ground-removed point cloud; ego is at the origin. Detections concentrate on true vehicles, with few spurious clusters — the precision-saturated behaviour reported in Table 4.2.

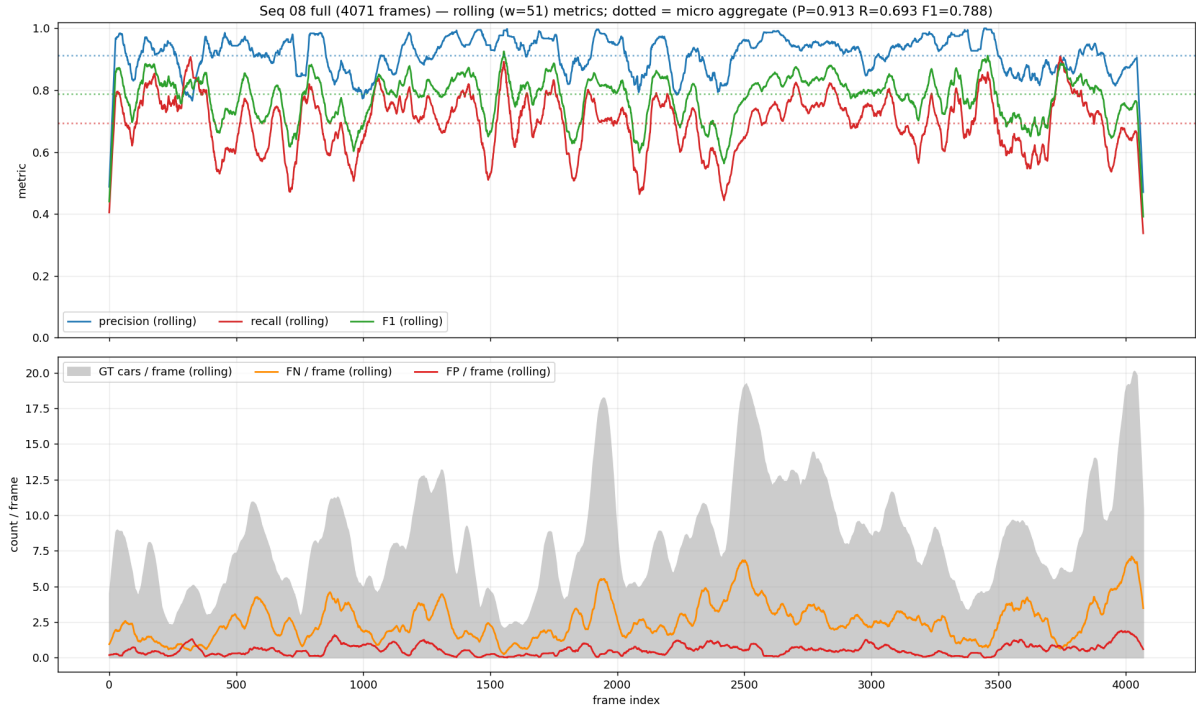


Figure 4.2: Per-frame detection behaviour over sequence 08. False positives (lower trace) stay low and flat — under one per frame — regardless of scene content, while per-frame recall tends to fall in frames with more ground-truth cars. The structural recall limit is root-caused in Section 4.7 (HDBSCAN splitting).

Table 4.3: Leave-one-sequence-out cross-validation over all eleven labelled sequences (full sequences, $\tau = 0.3$, final configuration). Each fold’s classifier trained on the other ten sequences and was never selected on the held-out one, so no row’s classifier has seen the sequence it scores. “Pooled” sums TP/FP/FN across folds; the excl.-seq00 pool isolates the residual seq-00 classical-tuning familiarity (Section 4.1).

Held-out seq	Precision	Recall	F_1	Mean IoU	TP / FP / FN
00 (4541 fr.)	0.913	0.761	0.830	0.932	42 763 / 4079 / 13 427
01 (1101 fr.)	0.951	0.336	0.496	0.953	539 / 28 / 1067
02 (4661 fr.)	0.793	0.700	0.743	0.903	10 919 / 2857 / 4685
03 (801 fr.)	0.733	0.623	0.673	0.904	1527 / 556 / 926
04 (271 fr.)	0.797	0.501	0.615	0.835	529 / 135 / 527
05 (2761 fr.)	0.861	0.731	0.791	0.910	10 671 / 1723 / 3927
06 (1101 fr.)	0.861	0.637	0.732	0.906	6374 / 1031 / 3637
07 (1101 fr.)	0.905	0.729	0.808	0.929	11 131 / 1162 / 4137
08 (4071 fr.)	0.881	0.737	0.803	0.912	25 733 / 3477 / 9189
09 (1591 fr.)	0.824	0.646	0.724	0.911	5141 / 1097 / 2813
10 (1201 fr.)	0.725	0.559	0.632	0.871	1832 / 694 / 1444
Pooled (all 11)	0.874	0.719	0.789	0.919	117 159 / 16 839 / 45 779
Pooled (excl. seq 00)	0.854	0.697	0.767	0.911	74 396 / 12 760 / 32 352
Macro mean $\pm$ std	0.840 $\pm$ 0.070	0.633 $\pm$ 0.121	0.713 $\pm$ 0.097	0.906 $\pm$ 0.030	—

Three things follow. First, model selection did not inflate the reported recall. The fold-08 classifier, blind to sequence 08, reaches recall 0.737 against the sequence-08 operating point’s 0.730 — if anything marginally higher, not lower — so the recall headline is not a

product of having selected the classifier on the reporting sequence. The model-selection optimism that does exist is confined to precision and is small: fold-08 precision is 0.881 versus the final 0.905 (an extra 801 false positives across 4071 frames), exactly the two-point, precision-only exposure the stage decomposition predicts (Section 5.2.2). The two sequence-08 rows are not two separate experiments but *one classifier under two checkpoint-selection rules*: the final model and the fold-08 model share the same architecture, the same ten training sequences (0 to 7, 9 to 10), and the same from-scratch 15-epoch regime, and differ only in which epoch is frozen — the final model takes the epoch that scored best on the sequence-08 validation set, the fold-08 model takes the last epoch and never inspects sequence 08 at all. The $0.730 \rightarrow 0.737$ (recall) and $0.905 \rightarrow 0.881$ (precision) deltas therefore isolate the effect of that selection: selecting on sequence 08 bought about two precision points and no recall. The isolation is not perfect — because sequence 08 cannot serve as the fold’s loss-monitoring split, the fold-08 model trains on 95 % of the final model’s clusters (a 5-percent monitoring slice is set aside) — but that confound removes training data from the fold-08 model, which would depress its recall, not inflate it, so it only strengthens the “recall was not inflated” reading. Second, the pooled cross-validated operating point over all eleven sequences — precision 0.874, recall 0.719, F_1 0.789, mean IoU 0.919 — sits within roughly one point of the sequence-08 operating point on every metric, so the reported numbers are representative of the labelled set and not specific to sequence 08. Third, the per-sequence spread is now explicit: recall ranges from 0.336 on sequence 01 (a sparse highway drive) to 0.761 on sequence 00 (dense urban), with a macro mean of 0.633 ± 0.121 ; precision is far steadier (0.840 ± 0.070). The gap between the pooled recall (0.719) and the macro mean (0.633) is driven by the smallest, sparsest sequences (seq 01 and seq 04, the latter only 271 frames), which the count-weighted pool down-weights and the unweighted mean does not; both figures are reported so neither hides the spread. Finally, the residual seq-00 classical-tuning exposure is bounded: dropping the seq-00 fold lowers pooled recall only from 0.719 to 0.697 (a 0.022 delta), so the historical seq-00 pipeline tuning does not drive the aggregate.

4.3.4 Detection Accuracy Versus Range

Recall is not uniform in range, and the way it varies is not the naive “far = worse” monotone one might expect. Table 4.4 stratifies detection by the range of each cluster’s centroid from the ego sensor, on a stride-20 sample of sequence 08 under the frozen configuration. **Deviation:** like the clustering benchmark, this table is *per-frame with the track filter off* (striding breaks the tracker), so its pooled recall (0.671) is a per-frame figure and does not restate the track-filtered headline 0.730; read the table as a range *trend*, not as a re-derivation of the headline.

Table 4.4: Detection stratified by range-to-ego on sequence 08 (stride-20 sample, frozen configuration, per-frame / track-filter-off). Recall is *non-monotonic*: it dips in the nearest bin, peaks at 10 m to 20 m, then falls with range. The nearest-bin dip is over-segmentation, the far decline is sparsity. Mean IoU is over matched detections. Summing the two near bins recovers the 0 m to 20 m figure ($R = 0.829$) comparable to the earlier range analysis.

Range (m)	Precision	Recall	F_1	Mean IoU (matched)	TP / FP / FN
0–10	0.896	0.787	0.838	0.902	311 / 36 / 84
10–20	0.855	0.862	0.859	0.926	432 / 73 / 69
20–40	0.762	0.549	0.638	0.934	400 / 125 / 329
40+	0.393	0.196	0.262	0.943	22 / 34 / 90
Pooled	0.813	0.671	0.735	0.923	1165 / 268 / 572

The counter-intuitive result is the nearest-bin recall dip: recall at 0 m to 10 m (0.787) is lower than at 10 m to 20 m (0.862), even though near cars carry far more points. This is not sparsity — it is over-segmentation. The split-rate analysis shows that HDBSCAN splits the majority of the very-near cars (a split rate of 66 % at 0 m to 10 m, falling to 4 % at 30 m to 50 m): a large, close car dissolves into several clusters, none of which matches the whole instance, so it is counted as a miss. The nearest bin also has the lowest mean IoU of matched detections (0.902), consistent with fragments eroding the overlap even on the cars that do match. We develop this splitting mechanism, and why post-hoc fragment-merging fails to recover it, in Section 4.7.

Beyond the peak the decline is ordinary sparsity: recall falls from 0.862 at 10 m to 20 m to 0.549 at 20 m to 40 m and 0.196 past 40 m, as cars thin to too few surviving points to cluster or to clear the 10-point eligibility rule (Section 4.2). Mean IoU of matched detections rises with range (0.902 $\rightarrow$ 0.943): the far cars that are detected at all are compact, single-cluster instances, so their overlap is clean even as their number collapses. The earlier precision-only, seq-00, pre-promotion range analysis is superseded by this on-frozen-config sequence-08 table and is retained only as a historical note.

4.4 Runtime Evaluation

Table 4.5 breaks the per-frame cost down by stage, measured over the full sequence 08 under the final (promoted) configuration (4068 timed frames, RTX 3070 Ti / Ryzen 7 7800X3D; per-stage figures from `timing_seq08_full_n4068_combined.json`). The breakdown shows where the time goes: the two classical geometric stages — ground-plane RANSAC and, above all, HDBSCAN clustering — dominate, together taking roughly 68 % of the frame; adding preprocessing and the geometric filter, the classical CPU stages account for about 94 %, leaving the single learned component — the classifier — at only ~ 5.4 % (the centroid/Hungarian tracker is a classical assignment step, not learned). The classifier is 33.7 ms for the ~ 45 clusters in a frame (~ 0.75 ms each) and the tracker is negligible.

Table 4.5: Per-frame runtime by stage on the full sequence 08 under the final (promoted) configuration (4068 timed frames; mean ms). Clustering alone is nearly 60 % of the frame. The total is ≈ 623.5 ms (≈ 1.6 fps) on this full run; a stride-20 sample of the same configuration estimates ≈ 650.6 ms (the two agree to within the sampling variation of the expensive stages). The pre-optimisation configuration totalled 921.3 ms. Percentages are of the 623.5 ms total.

Stage	Mean (ms)	Share
Data load	0.7	0.1%
Preprocessing (z-filter, denoise, voxel)	125.9	20.2%
Ground removal (RANSAC)	56.2	9.0%
Clustering (HDBSCAN)	370.4	59.4%
Geometric filter	36.2	5.8%
Classification	33.7	5.4%
Tracking	0.3	0.1%
Total	623.5	100% (≈ 1.6 fps)

The promoted configuration is itself the product of a runtime optimisation that targeted the dominant classical stages directly: reducing the RANSAC iteration count and coarsening the clustering voxel to 0.10 m cut the per-frame total by roughly a third, from a pre-optimisation 921.3 ms to 623.5 ms on the full sequence (650.6 ms on the stride-20 sample), and the coarser voxel also improved detection — it is the promoted configuration used throughout this chapter. A sub-400 ms target was shown to be unreachable through the authorised optimisation tiers — the post-ground object cloud is intrinsically sparse, so coarser voxelisation compresses it little and the Python HDBSCAN retains a floor of ~ 185 ms — and the achieved operating point was accepted, real-time operation being out of scope (user decision, 2026-07-23). Point-completion, when it runs, adds ~ 13 ms per completed car on top of detection and is not part of the detection budget above.

4.5 Completion Results

4.5.1 Validation of the Donor Metric

Before using it we required the metric to pass four pre-stated checks (Table 4.6); it passes all four.

Table 4.6: The four validation checks for the donor metric, and the observed outcome (seq 08, per-car medians, $n = 39$; the metric’s original acceptance run, on the production path as it stood before the Section 3.3 length prior). Coverage figures are the car-median `cov@0.1`; the guard figures are car-median out-of-box fractions.

Check	Outcome
(a) The raw partial must rank last on coverage (it covers no novel surface by construction).	Passes at every τ : raw 0.000 < mirrored 0.043 < completed 0.304.
(b) Per-car scores must be stable enough for a per-car median to mean something.	Median per-car coverage IQR 0.14 — moderate, as expected from varying view geometry; acceptable for medians.
(c) The ranking must not depend on the threshold τ .	Stable over $\tau \in \{0.10, 0.15, 0.20\}$: completed coverage 0.345 / 0.304 / 0.281; the order never inverts.
(d) Completion must not win by hallucinating: its out-of-box fraction must not exceed the mirrored baseline.	Passes: completed out-of-box $0.0003 \leq$ mirrored 0.0083 (center/heading error makes the mirror spill more).

Check (d) is the one that makes the coverage number meaningful: without it, a model could raise coverage simply by emitting surface in every direction. The mirrored baseline in checks (a) and (d) is deliberately adversarial — it is a real way to add surface with zero learned parameters, so completion earns its place only by beating it.

The Per-Band Hallucination Guard

The guard as first designed — check (d) above — was a single median pooled over all cars, and that pooling hid a real failure mode. When we later evaluated a geometry change (the fixed length prior of Section 3.3) with this guard — now re-run on the promoted configuration, whose well-observed static set is $n = 40$ (8 + 27 + 5 across the compact/normal/long bands) — the pooled out-of-box fraction stayed comfortably low (0.0020) and the change appeared safe. Splitting the same guard by car length told a different story (Table 4.7): on the compact band the length prior over-extended cars to 0.0433 out-of-box — about 100 $\times$ that band’s own mirrored baseline — while 32 of 40 cars sitting near 0.0015 kept the pooled median quiet.

Table 4.7: The pooled guard is blind to a band-localised violation. Out-of-box fraction as completed / that band’s mirrored baseline, split by amodal-GT length. The final length prior passes the pooled check yet hallucinates on compacts at 100 $\times$ the band baseline.

Config	compact <3.6 m	normal	long ≥ 4.6 m	pooled
prior off	0.0003 / 0.0004	0.0008 / 0.0129	0.0000 / 0.0292	0.0004 ✓
length prior on	0.0433 / 0.0004	0.0017 / 0.0129	0.0000 / 0.0292	0.0020 ✓

Two lessons follow, and both are now built into the metric. First, the guard is computed *per length band against that band’s own mirrored baseline*, not as a pooled median; a change is only accepted if it clears the guard in every band. Second, on the compact band the mirrored baseline is itself near zero (0.0004): reflecting an already-short car

barely leaves its box, so the bar there is effectively “do not leave the box at all,” which no configuration clears. We therefore report the guard as a *ratio* to the band baseline rather than a pass/fail bit, because the ratio (for example, a $16\times$ band violation reduced from an incumbent $100\times$ — a roughly $7\times$ reduction in hallucination) is the informative quantity, while the binary bit is unclearable by construction and would hide genuine progress.

This is a property of the measurement, not a retraction of a result: the direction of the geometry change it was auditing still holds; what changed is that the guard now reads at the resolution where the failure lives. Any future change to the completion geometry is evaluated per band under this refined guard.

Section 3.4 established a metric that measures, on real cars, how much previously-unseen surface a completion recovers, and showed that it passes an explicit validation battery. This part of the chapter uses it. It reports three things the completion method was built to deliver and one it was not: that the completion adds genuinely unseen surface (Section 4.5.2), that this translates into a better amodal bounding box (Section 4.5.3), that moving cars — which the static-car metrics cannot score — complete no worse than static ones (Section 4.5.4), and that both headline claims survive a pre-registered replication on a held-out sequence (Section 4.5.5). Throughout, the unit of analysis is the car: because the frames of one parked car are autocorrelated, each car is reduced to its per-car median and significance is tested across cars with the Wilcoxon signed-rank test, the paired design of Section 3.4.

4.5.2 Real-Data Surface Coverage

The primary completion result is that PCN reconstructs surface the input frame never observed. Table 4.8 reports the donor metric on 39 of seq 08’s 40 well-observed static cars (one car’s pairs are all gate-skipped), scoring three clouds — the raw partial, its symmetry mirror, and the completion — against each car’s donor-only novel set. This is the metric’s acceptance run, taken on the production path before the Section 3.3 length prior; the length geometry is evaluated in Section 4.5.3.

Table 4.8: Real-data surface coverage on the donor metric (seq 08, per-car medians, $n = 39$ of the 40 well-observed static cars — one car’s pairs are all gate-skipped; $\tau = 0.15$ m). **cov@0.1** is the fraction of never-observed (donor-only) surface recovered within 0.1 m; median novel distance is its threshold-free companion; out-of-box is the hallucination guard. All three pairwise contrasts are significant at $p < 10^{-6}$.

Cloud	cov@0.1 $\uparrow$	med. novel dist. (m) $\downarrow$	out-of-box $\downarrow$
raw partial	0.000	0.518	0.000
mirrored	0.043	0.332	0.008
completed	0.304	0.161	0.000

The completion recovers about 30% of never-observed surface within 0.1 m — *seven times* the symmetry-mirror baseline — while its out-of-box fraction stays at the raw partial’s floor, so the coverage is real surface, not surface invented everywhere. The mirror is the informative comparison: reflecting the input across the car’s symmetry plane is a real, zero-parameter way to add surface, and the learned model earns its place only by beating it, which it does more than sixfold. This is the project’s first positive real-data

evidence that completion adds unseen structure, and it complements the box-level value of Section 4.5.3.

Two breakdowns qualify the headline. First, coverage is uneven over the car: the occluded *far side* is well recovered (completed cov 0.321) and the roof reasonably so (0.203), but the *far end* — the portion of the car truncated by the single viewpoint — is the weakest region (0.133). The far end is exactly where a one-sided partial is normalised around its observed near portion, and it is what the length geometry of Section 4.5.3 sets out to move. (The mirror scores essentially zero on the far end by construction — a reflection across the length plane cannot add end surface — which is a built-in sanity check on the region split.) Second, the coverage is robust to input sparsity: completed cov@0.1 holds at 0.30–0.33 from the sparsest (<100 points) to the densest (≥ 300) inputs, while the mirror falls from 0.083 on the densest inputs to 0.010 on the sparsest, as the reflection of a sparse cluster leaves ever more of the car uncovered. Completion helps most, relative to the cheap baseline, exactly where the sensor saw least.

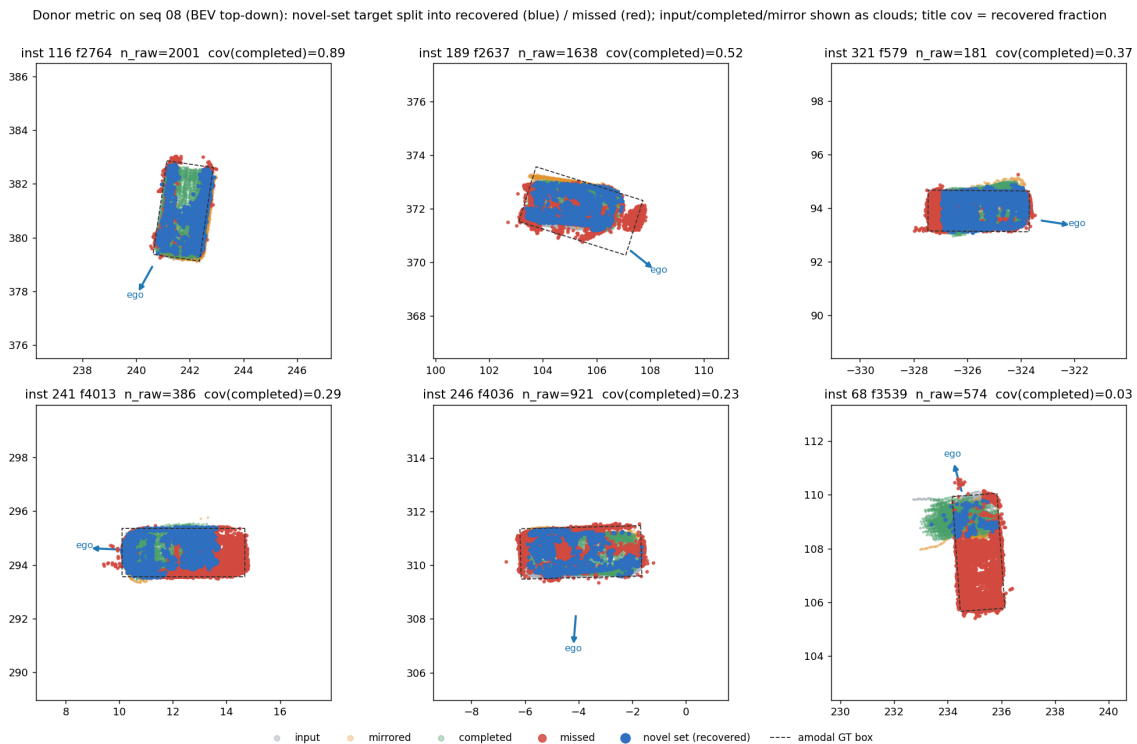


Figure 4.3: The donor metric in bird’s-eye view on six seq-08 cars, spanning the coverage range from best (top-left) to a residual failure (bottom-right). The *novel set* — the donor-only surface the input frame never saw — is drawn blue where the completion recovers it within 0.1 m and red where it does not, so the blue fraction of the target is exactly the reported cov@0.1 in each title. The three candidate clouds are shown faintly for context: the input partial (grey), its symmetry mirror (orange), and the completion itself (green). The upper panels close the occluded far side almost entirely; the last isolates the residual failure modes — far-end under-completion and heading/centre error on diagonal or sparse views — that Section 4.5.3 and Section 3.3 address.

Comparison with Reference-Free Completion Metrics

The donor metric earns its place only if the field’s existing reference-free measures cannot already do the job. The established ones — the unidirectional Chamfer and Hausdorff distances (UCD, UHD; Section 2.1.4) — score a completion by how closely it reproduces its own partial *input*. Computed on the same 39 cars (per-car medians, input $\rightarrow$ cloud direction), they invert the ranking the task calls for (Table 4.9): the raw partial and its mirror score a perfect 0 — any cloud that contains the input points is an optimum — while the completion scores worst, penalised for every point it adds. The gap is not marginal: the raw partial’s median forward UCD is 0m against the completion’s 0.090m ($p < 10^{-11}$, Wilcoxon across cars). Yet the completion is the only cloud that recovers unseen surface, as the donor coverage column (repeated from Table 4.8) shows. UCD and UHD measure input fidelity, so their optimum is the completion that does nothing; this is why the unpaired-completion literature pairs them with a separate plausibility term, and why a metric restricted to the never-observed surface is needed here.

Table 4.9: The standard reference-free metrics against the donor metric (seq 08, 39 cars, per-car medians). UCD/UHD are computed input $\rightarrow$ cloud (lower is “better”: closer reproduction of the input), so the raw partial and mirror — which contain the input — are their trivial optima and the completion is scored worst. Donor cov@0.1 (higher is better) is the only column that credits recovered unseen surface.

Cloud	UCD (m) ↓	UHD _{max} (m) ↓	UHD _{p95} (m) ↓	donor cov@0.1 ↑
raw partial	0.000	0.000	0.000	0.000
mirrored	0.000	0.000	0.000	0.043
completed	0.090	0.286	0.183	0.304

4.5.3 Downstream Utility: The Amodal Bounding Box

Surface coverage shows the completion is geometrically faithful; the box evaluation shows it is useful. For each true-positive (frame, car) pair on the well-observed static cars we fit an oriented box to the raw partial and to the completion with the same fitter used to build the amodal ground-truth boxes (so fitter bias cancels), and compare each against the GT box. This is the pre-registered “completion adds value” test. Table 4.10 reports it.

Table 4.10: Amodal box quality, raw partial vs. completion (seq 08, per-car medians, $n = 39$ of the 40 well-observed static cars — one has no completed pair). BEV IoU is the oriented bird’s-eye-view box overlap; $|\Delta W|, |\Delta H|, |\Delta L|$ are absolute dimension errors; centre error is in the ground plane. p is the Wilcoxon signed-rank across cars. These figures use the original detection configuration; the width, height, and overlap wins reproduce under the promoted configuration of Table 4.16 (BEV IoU 0.725→0.747, $|\Delta W|$ 0.210→0.122, $|\Delta H|$ 0.227→0.144).

Metric	Raw	Completed	Wilcoxon p	
BEV IoU $\uparrow$	0.707	0.747	1.9×10^{-3}	better
$ \Delta W $ (m) $\downarrow$	0.270	0.170	1.5×10^{-4}	better
$ \Delta H $ (m) $\downarrow$	0.255	0.131	1.6×10^{-10}	better
centre err. (m) $\downarrow$	0.286	0.234	2.8×10^{-5}	better
$ \Delta L $ (m) $\downarrow$	0.447	0.456	0.65	neutral
yaw err. (deg) $\downarrow$	3.5	3.0	0.74	neutral

Completion improves the box on width, height, centre, and overall BEV overlap with strong significance, and nothing degrades significantly. The improvements are largest where the sensor saw least — BEV IoU by input size rises 0.461→0.599 on the sparsest cars (<100 points), 0.585→0.678 at 100–300, and 0.703→0.744 above 300 — which is the right profile for the use case, since a well-observed car needs little help. The width gain is not an artefact of the GT accumulation: it holds both for the 13 cars whose width the accumulation genuinely constrains (both flanks eventually seen: 0.291→0.182) and for the other 26 (0.167→0.118).

Length: a regression the geometry corrected. In Table 4.10 length is neutral, which was too generous a reading of an untreated defect: a partial truncated at the occluded far end completes short, so it under-extends length. The completion’s length geometry (Section 3.3) corrects this — on the promoted configuration the final completed box reaches median BEV IoU 0.771 against the raw partial’s 0.725, with $|\Delta L|$ improved to 0.304 and centre error to 0.184. The ablation of the length prior that establishes this is reported with the completion ablations (Section 4.6.2).

4.5.4 Moving Cars: A Plausibility Check

Both metrics so far are defined on *static* cars by construction: a paired amodal reference needs a stationary target. Production nonetheless completes moving cars, and those completions were never validated. Because completion runs on a single reference frame’s cluster — not on accumulated points — there is no motion smear in its input, so there is a mechanistic reason to expect motion not to matter; this section checks it. On the final seq-08 output (518 completed car tracks) each track is labelled static, moving, or ambiguous by the net horizontal displacement of its centroid (a label-free proxy, since the output tracks are unlabelled), and each completion is scored by a size-plausibility recipe (bird’s-eye dimensions falling in the car range). Table 4.11 gives the result.

Seq 08: amodal GT vs raw-partial vs completed boxes (BEV, world X-Z)
gray = raw cluster, light green = completed points

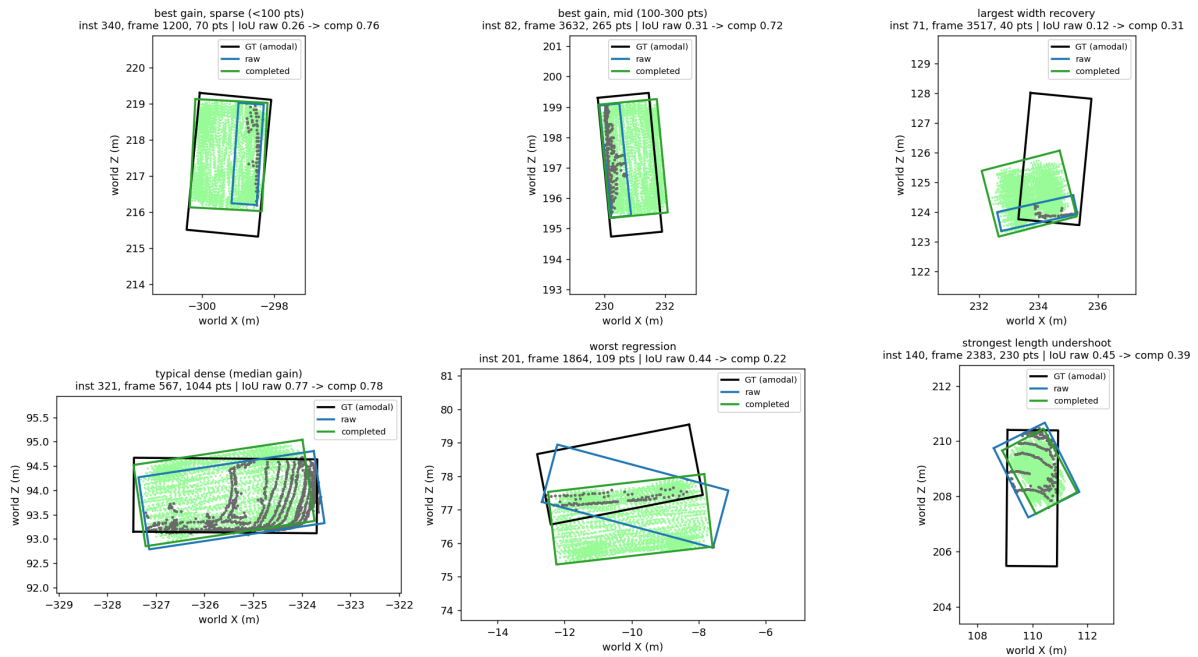


Figure 4.4: Amodal box overlays on six seq-08 cars: the ground-truth box (black), the box fitted to the raw single-frame partial (blue), and the box fitted to the completion (green), over the raw points (grey). The completion tightens the box on the occluded width and far end and centres it better; the one strong regression panel is a heading error on a sparse (109-point) input, a diagonal-view failure mode also visible in Figure 4.3.

Table 4.11: Completion plausibility by track motion (final seq-08 output, 518 completed car tracks). Motion is the net displacement of the track centroid (median of first vs. last five frames). Plausible = bird’s-eye dimensions in the car range ($L \in [3.3, 4.9]$, $W \in [1.5, 2.1]$, $H \in [1.1, 1.7]$ m).

Group (net displacement)	n	plausible	rate	median $L/W/H$ (m)
static (≤ 2 m)	419	225	53.7 %	3.81 / 1.94 / 1.49
moving (≥ 5 m)	19	11	57.9 %	3.84 / 2.05 / 1.44
ambiguous (2 m to 5 m)	80	46	57.5 %	3.95 / 1.92 / 1.49
all	518	282	54.4 %	3.83 / 1.94 / 1.49

Moving cars complete *no worse* than static ones — slightly higher, in fact (57.9 vs. 53.7 %), well within noise at $n = 19$ — with near-identical median dimensions, and every motion bucket clusters at 54 % to 58 %. Motion does not degrade completion plausibility, as the single-frame input predicts. One honest caveat about the absolute level, not the comparison: the plausibility recipe uses axis-aligned dimensions, which overestimate width and length for cars oriented diagonally to the world axes, so the ~ 54 % rate understates completion quality (most failures are inflated width on genuine car-shaped footprints). Since both groups use the identical recipe, the static-vs-moving conclusion is unaffected. The value of the check is that the statics-only design of the earlier sections does not hide a mover failure mode.

4.5.5 A Pre-Registered Held-Out Replication

The completion constants — the width and length priors, the scale correction, the gate thresholds — were all developed on seq 08. A fair test of whether the method generalises therefore has to run on a sequence that took no part in that development, under a protocol fixed in advance so the outcome cannot be reinterpreted after the fact. We pre-registered one (`preregistration_heldout.md`, committed with an immutable timestamp before any held-out run). The held-out sequence is seq 00 (seq 05 had too few well-observed static cars to test); the two primary metrics were fixed as BEV box IoU (Section 4.5.3) and donor `cov@0.1` (Section 4.5.2), each requiring the completion to beat the raw partial with $p < 0.05$; three refutation rules (fails to generalise / seq-08 specific length constants / an uncovered result) and a three-level outcome taxonomy were registered alongside. Nothing was retuned for seq 00.

One confound is logged rather than hidden: seq 00 is in the classifier’s training split, so its *detection* recall reads optimistically. This does not touch the completion claim, which is a paired raw-vs-completed comparison on identical true-positive inputs, and PCN was trained only on synthetic data and never saw seq 00. Table 4.12 gives the outcome on the 45 well-observed static cars.

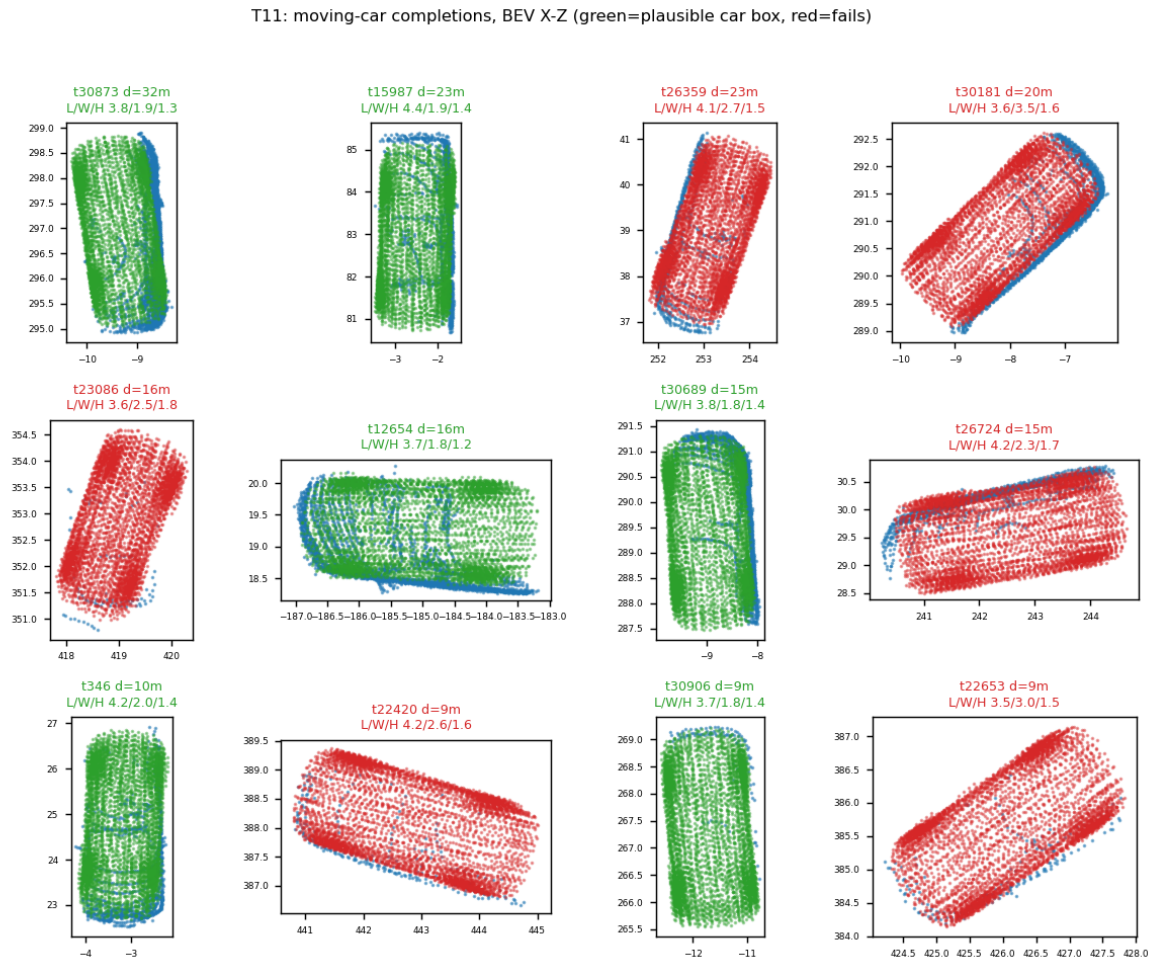


Figure 4.5: Bird's-eye completions of moving-car tracks on seq 08 (blue partial input, green completion). Most form recognisable car footprints; the failures are width inflation on diagonally-oriented cars, an artefact of the axis-aligned plausibility measure rather than of motion.

Table 4.12: Held-out replication on seq 00 (well-observed static cars, per-car medians, $n = 45$; the two pre-registered primaries in bold, secondary dimensions below). Both primaries are decisive wins over the raw partial.

Metric	Raw	Completed	Wilcoxon p
BEV IoU $\uparrow$	0.739	0.766	1.6×10^{-3}
donor cov@0.1 $\uparrow$	0.000	0.413	≈ 0
$ \Delta W $ (m) $\downarrow$	0.203	0.165	4.0×10^{-4}
$ \Delta H $ (m) $\downarrow$	0.278	0.120	7.8×10^{-9}

Both headline claims replicate: on a sequence that played no part in tuning, the completion beats the raw partial on box overlap ($0.739 \rightarrow 0.766$, $p = 1.6 \times 10^{-3}$) and on surface coverage ($0.000 \rightarrow 0.413$, $p \approx 0$), with the secondary width and height errors improving as they did on seq 08. Applying the pre-registered rules verbatim, the arbitration verdict was **partially holds**: neither generalisation rule triggered — both primaries won and no testable length band regressed against its seq-08 level — but seq 00’s well-observed set contains no long cars (≥ 4.6 m), so the long-band predictions and the long-band hallucination guard are simply untestable on it. The downgrade from “holds” to “partially holds” is therefore a *coverage gap in the held-out data, not a weak result*: everything that could be tested passed, and the one thing that could not be tested is a property of seq 00’s car population, not of the method.

4.6 Ablation Studies

This section decomposes both halves of the system into the contribution of each component, in two groups: detection ablations (Section 4.6.1) and completion ablations (Section 4.6.2). Every ablation is produced under the frozen protocol of Section 4.2 unless a deviation is stated explicitly.

4.6.1 Detection Ablations

Stage decomposition: classifier and track-filter contributions. Table 4.13 isolates the two filtering stages that sit on top of geometric clustering. The geometric-only row uses clustering plus the geometric size/shape filters with no learned classifier and no track filter (like-for-like under the promoted configuration); the full row is the headline.

Table 4.13: Stage ablation on the full sequence 08 (frozen configuration). Rows add one filtering stage at a time on top of geometric clustering. The per-frame classifier is the precision mechanism (it removes $\sim 149\,000$ junk clusters) but is conservative on borderline cars (recall drops); the offline track filter then recovers recall by temporal voting while raising precision further. Mean IoU is over matched detections only. Geometric-only and the middle row are per-frame (no track filter); the full row uses the offline track filter.

Configuration	Precision	Recall	F_1	Mean IoU	TP / FP / FN
Geometric only	0.149	0.775	0.250	0.907	27 051 / 154 612 / 7871
+ classifier	0.820	0.677	0.741	0.921	23 629 / 5204 / 11 293
+ classifier, track filter	0.905	0.730	0.808	0.912	25 478 / 2676 / 9444

The decomposition tells a two-step story. The **classifier is the precision mechanism**: geometric clustering under the promoted coarse-voxel setting emits many mostly-junk clusters (precision 0.149), and the per-frame classifier removes about 149 000 of them across the sequence (FP 154 612 $\rightarrow$ 5204), lifting precision from 0.149 to 0.820. It pays for this with recall — being conservative, it discards borderline true clusters too, so recall falls from 0.775 to 0.677. The **track filter then recovers that recall while improving precision again**: linking detections across frames and accepting only consistent tracks lifts precision to 0.905 and recall to 0.730 (TP 23 629 $\rightarrow$ 25 478), because a car seen across many frames is kept even where a single frame classified it as unknown, while transient false positives fail the track-length test. The net of both stages is still a recall trade — geometric-only recall (0.775) exceeds the full-pipeline recall (0.730), roughly 1600 true positives forgone for the $\sim$ 152 000-false-positive removal — but that trade runs through a recall dip and a partial recovery, not a simple monotone loss. We report it rather than hiding it: the system is deliberately tuned for precision, and the recall it forgoes here is small next to the recall it never had (Section 4.7). The pre-promotion geometric-only figure (F_1 0.305) is superseded by this like-for-like 0.250 and kept only as a historical record.

Redundancy of synthetic pretraining (Stage A ablation). The classifier ships as a binary car/not-car network trained *from scratch* on mined real clusters (Stage B). A natural alternative is a two-stage recipe in which a synthetic-ShapeNet Stage A pretraining seeds Stage B; we tested whether that pretraining is necessary and found it is not — a negative result we report as a finding, because it changes the recommended recipe.

At the classifier level, on the sequence 08 validation clusters, the from-scratch network matches the pretrained one: macro F_1 0.9285 from scratch versus 0.9225 pretrained (the two are within run-to-run noise, with scratch marginally ahead). At the pipeline level a small precision edge for the pretrained checkpoint appeared on a 100-frame sample (FP 27 versus 62), but it was uncontrolled (different training seed and code) and **did not survive at full scale**: on the full sequence 08 the two checkpoints are identical in F_1 to three decimals. We therefore drop Stage A from the production pipeline and keep it as ablation material.

The reason the synthetic prior is redundant — and cannot simply be added for safety — is a total, symmetric domain gap between synthetic and real point clusters. Table 4.14 gives the cross-domain transfer matrix: a model trained on one domain and evaluated on the other scores a car-class F_1 of 0.000 in both off-diagonal directions, and fine-tuning on real data erases whatever the synthetic pretraining had learned. Given 420 000 real training clusters, the real data does essentially all the work; the synthetic prior neither helps nor, once fine-tuned, survives.

Table 4.14: Cross-domain classifier transfer (car-class F_1). Each model is trained on one domain and evaluated on the other’s validation set; diagonal cells (italic) are in-domain, for scale. Both off-diagonal cells collapse to 0.000 — against strong in-domain performance (0.999, 0.885) — so synthetic and real clusters are mutually out-of-distribution and synthetic pretraining transfers nothing to real inference.

Trained on	Evaluated on	
	Synthetic	Real
Synthetic (Stage A)	<i>0.999</i>	0.000
Real (Stage B)	0.000	<i>0.885</i>

Clustering method: cost and benefit of HDBSCAN. The clustering stage is HDBSCAN. Because it is by far the most expensive stage (Section 4.4), we benchmarked it against two cheaper density/connectivity clusterers — DBSCAN and Euclidean-tolerance clustering — at the same voxel resolution and under an identical pipeline (Table 4.15). **Deviation:** this benchmark is per-frame with the track filter off, because it strides across the sequence and the tracker requires contiguous frames; the numbers are therefore comparable to each other but not to the track-filtered headline.

Table 4.15: Clustering-method benchmark at fixed voxel resolution (0.10 m), identical pipeline, per-frame / track-filter-off. HDBSCAN wins F_1 on both sequences, entirely through recall; precision is essentially method-independent. The alternatives are 5–7× faster. Mean IoU is over matched detections; the timing column is the mean per-frame clustering time.

Seq	Method	Precision	Recall	F_1	Mean IoU	Clustering (mean ms)
00	HDBSCAN	0.962	0.731	0.831	0.968	391
	DBSCAN	0.962	0.637	0.767	0.964	59
	Euclidean	0.967	0.636	0.768	0.982	78
08	HDBSCAN	0.813	0.671	0.735	0.923	451
	DBSCAN	0.848	0.565	0.678	0.927	69
	Euclidean	0.838	0.576	0.683	0.950	93

The gap is entirely a recall gap — precision is within 0.035 across all methods on either sequence, while HDBSCAN’s recall exceeds the alternatives by 9 to 11 points. An ε -sweep confirms no fixed radius closes it: the best fixed-radius setting (Euclidean at $\varepsilon = 0.4$ m) reaches F_1 0.800 on sequence 00, still short of HDBSCAN’s 0.831. HDBSCAN’s variable-density criterion recovers cars that a single radius cannot, which is exactly the recall margin that matters here. We therefore **adopt nothing** from the benchmark and keep HDBSCAN in production, paying its runtime cost knowingly; DBSCAN and Euclidean remain available as command-line options for reproduction.

4.6.2 Completion Ablations

These ablations isolate the effect of each completion-pipeline choice on the final output; except where noted they vary one component of the completion path (Section 3.3) and re-score the same detections, with the completion network itself unchanged.

Inference normalisation. PCN is trained in a canonical car frame, so a real partial must be mapped into that frame before completion (Section 3.3.4). Ablating that step against a naive alternative — aligning the partial with a 3D PCA frame and normalising by its own centroid and radius — isolates its effect on the same checkpoint (Figure 4.6): the canonical-frame normalisation lifts the fitted box’s BEV IoU at every input density (0.32→0.73 sparse, 0.25→0.74 mid, 0.47→0.85 dense), while the naive frame leaves the box mis-oriented and mis-scaled off the ground-truth box.

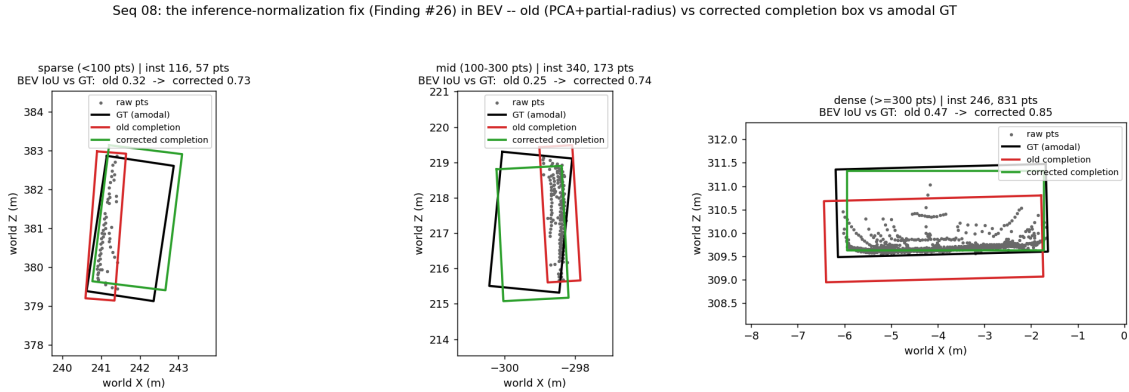


Figure 4.6: Canonical-frame versus naive normalisation in bird’s-eye view at three input densities; *same checkpoint*, only the normalisation differs. Each panel fits an oriented box to the completed cloud and compares it to the amodal ground-truth box (black) over the raw points (grey). A naive PCA + partial-radius normalisation (red) is mis-oriented and mis-scaled — the 3D PCA picks an arbitrary principal axis rather than gravity — while the canonical-frame box (green) aligns with the GT, lifting BEV box IoU at every density: 0.32→0.73 (sparse), 0.25→0.74 (mid), 0.47→0.85 (dense) — per-panel values, not an aggregate finding. The figure measures pose and scale via the box; the completed cloud is car-shaped in the canonical frame but filled rather than crisp (Section 4.5.1 quantifies real-data surface coverage).

Input-quality gate. The L-shape gate decides whether a detected car reaches completion as a clean single-object cluster. An eleven-sequence ablation switches it off and then on and scores each output by the plausibility box (the footprint length/width/height check): the fraction of plausible completions rises from 0.473 to 0.686 pooled, a 0.213 shift positive on every one of the eleven sequences (per-sequence 0.081 to 0.319). The gate carries a measured cost, reported rather than hidden: it drops 15.5 % of otherwise-plausible cars (356 pooled) as false rejections. On the same clean-gated tracks the completion precision rises from 18/47 (38 %) to 18/26 (69 %).

Length geometry. Completion under-extends the occluded far end — the one box dimension the raw-versus-completed test of Section 4.5.3 rated neutral. Table 4.16 ablates the longitudinal centre push on the promoted configuration ($n = 40$): the completion with no length correction, the fixed 4.14m population prior, and the final per-car estimate, each against the raw partial.

Table 4.16: Length geometry on the amodal box (seq 08, well-observed static cars, per-car medians, $n = 40$, promoted configuration). “no prior” is the completion with the canonical-frame centre but no longitudinal push; “fixed 4.14” is the population-median prior; “per-car” is the final 90th-percentile track estimate. The compact row is signed ΔL on cars < 3.6 m ($n = 8$), where a fixed prior over-extends.

Metric	Raw	no prior	fixed 4.14	per-car (final)
$ \Delta L $ (m) $\downarrow$	0.428	0.476	0.354	0.304
BEV IoU $\uparrow$	0.725	0.743	0.747	0.771
centre err. (m) $\downarrow$	0.271	0.227	0.229	0.184
compact signed ΔL (m)	−0.007	—	+0.295	+0.063

Two things are visible here, and both are stated as the honest picture rather than the flattering one. First, Table 4.10’s neutral $|\Delta L|$ averages over- and under-completion and so hides a real regression: the *signed* length shows the completion systematically under-extends the occluded far end (normal cars $-0.485 \rightarrow -0.545$). On the promoted configuration this surfaces even in the absolute $|\Delta L|$, where completion without a length prior makes box length worse than the raw partial ($0.428 \rightarrow 0.476$, $p = 0.027$), which the far-end truncation causes. The fixed prior reverses it (0.354) and the per-car estimate improves it further (0.304) while lifting BEV IoU to 0.771 and cutting centre error to 0.184. Second, moving from the fixed prior to the per-car estimate is what removes the over-extension of genuinely compact cars: their signed ΔL goes from 0.295 (the fixed median over-extends a short car) to 0.063 ($p = 0.016$ at $n = 8$), with the same fix showing up in the per-band hallucination guard of Section 3.4.

The trade-off, stated plainly. The per-car estimate buys this box accuracy at a measured cost in donor coverage: pooled completed cov@0.1 falls $0.403 \rightarrow 0.364$ and far-end coverage $0.346 \rightarrow 0.316$ relative to the fixed prior. Per band it is better on compact and long cars and worse on normal ones. The two metrics genuinely disagree here, and a control run settled which reading is right: deliberately over-extending (90th percentile $+0.45$ m) improved both coverage and the box on normal and long cars, which means those completions are still honestly too short rather than the metric being fooled. In other words the per-car estimate trades a little pooled coverage — driven by the normal band it slightly shortens — for a correctly-sized box across all bands, and the residual shortfall is a real limitation, addressed next.

A bounded negative on the residual shortfall. Targeting each car’s true length still leaves completions short on normal and long cars, which pointed to a second under-extension mechanism inside the completion itself (PCN under-fills its normalised frame, and the centre push doubles as an output rescale). It was investigated under pre-registration (the “closed negative”): decoupling the scale from the length push and adding a calibrated fill factor did fix the target (normal-car $|\Delta L|$ $0.329 \rightarrow 0.232$) but over-extended compacts and failed the pre-registered compact non-regression guard on both test sequences. The verdict was to reject it; the knobs stay in the code defaulted off. It is reported here because it bounds the result: the residual far-end shortfall on long cars is a characterised, length-dependent limit, not an oversight.

Completion architecture (PCN vs. PoinTr). Swapping the completion network holds the rest of the pipeline fixed. On held-out synthetic cars PoinTr’s edge over PCN is small (Chamfer 0.161→0.153 m), and on the same clean-gated real tracks the two are statistically indistinguishable (18/26 versus 16/26 plausible completions, the difference within threshold noise), because the residual centroid-estimation gap is architecture-independent. PCN is therefore kept off the shelf; the completion result is set by input quality and geometry, not by the choice of network.

4.7 Result Analysis and Error Analysis

The detection operating point above established two facts: the pipeline is precision-saturated (false positives are low and flat, about 0.7 per frame), and it is recall-limited, missing roughly one car in four ($R = 0.730$ on sequence 08). This section explains why the recall is limited, and it also underpins the completion results reported earlier (Section 4.5): the same clustering step that caps recall is the one that decides whether a detected car reaches completion as a clean single-object cluster. We show that the shortfall is a clustering failure rather than a classification failure (Section 4.7.1), root-cause it to HDBSCAN splitting large, close cars into fragments (Section 4.7.2), report a search of repair strategies that all fail to generalise (Section 4.7.3), and describe the one intervention that did help — coarsening the clustering resolution — which forces a correction of an earlier, overstated “hard ceiling” claim (Section 4.7.4). The negative results are reported as findings, not apologised for: mapping which repairs fail, and why, is itself a finding worth reporting, and it is what justifies the residual limitation carried to Chapter 5. Unless stated otherwise, the recall denominator throughout is the one defined in Section 4.2 — ground-truth car instances with at least 10 points surviving preprocessing.

4.7.1 The Shortfall as a Clustering Failure

The recall ceiling is upstream of the learned classifier. The stage decomposition of Section 4.6 makes this direct: geometric clustering with the size/shape filters but no learned classifier and no track filter already reaches recall 0.775, which is higher than the full pipeline’s 0.730. Every learned stage that follows can only hold or reduce that recall: the classifier discards clusters, and the centroid tracker only re-labels and filters clusters that geometric clustering has already produced — it emits a detection in a frame only where a cluster exists in that frame, and never interpolates a detection across a gap, so neither stage can invent a detection where clustering found nothing. Across the full sequence the two stages together trade roughly 1600 true positives (27 051 geometric-only versus 25 478 full) for the removal of $\sim 152\,000$ false positives; this is a distinct trade from the +1655 true positives that the resolution change of Section 4.7.4 later recovers. Whatever caps recall is therefore already present in the clusters that geometric clustering produces, before the classifier sees them.

The geometric filters are not the cap either. Table-level ablation of the size/shape gates (sequence 00, 100 frames) shows that of the 16 454 HDBSCAN clusters in the sample, only 2651 (16.1 %) pass the filters, and 1827 of the rejected clusters do overlap a ground-truth car — dominated by `min_volume` (67.9 % of the GT-matching rejections) and `min_points` (26.4 %); sequence 08 shows the same pattern (66.5 % / 28.0 %). The dominant gates are `min_volume` and `min_points` — the two that reject small clusters. This is consistent with the rejected clusters being the smaller sub-fragments of cars that HDBSCAN has already

split (the dominant fragment passing while the splinters fail the size gates), which the split analysis of Section 4.7.2 then confirms is the prevailing failure mode. We state this as an inference rather than a direct measurement: the ablation records only that a rejected cluster overlaps some ground-truth car, not whether that same car also retains a surviving cluster, so it establishes that the rejections are dominated by small fragments, not that every one belongs to an otherwise-detected car. Either way the diagnosis points one stage further upstream, at the clustering itself.

4.7.2 Root Cause: HDBSCAN Fragmentation of Large, Close Cars

To quantify how clustering caps recall we measured, for every eligible ground-truth car, how its points distribute across HDBSCAN clusters. Each car is assigned to exactly one of four states that partition the population: *ok* (one dominant cluster, under 30 % noise), *split* (points spread over two or more clusters), *noisy* (one cluster but at or above 30 % noise), or *all-noise*. A separate, cross-cutting flag marks merged cars (a car whose dominant cluster is shared with another car); a merged car still has a dominant cluster, so it is not a fifth partition class and is reported below as a memo count rather than a table row that would be summed. Table 4.17 gives the result on both sequences.

Table 4.17: Distribution of ground-truth cars across HDBSCAN clusters (pre-promotion configuration; sequence 00, 100 frames; sequence 08, held-out sample). The four states partition the population and sum to the denominator; split is the dominant failure mode and merging (the cross-cutting memo row, not summed) is negligible. Split cars are uneven — the largest fragment holds a median 0.92 of the points on sequence 00 and 0.77 on sequence 08 — but a large point-fraction does not guarantee a match (see text): recall shows most split cars are lost.

Status	Seq 00 (1631 cars)		Seq 08 (811 cars)	
	Count	%	Count	%
ok (single dominant cluster)	1117	68.5	509	62.8
split (2+ clusters)	514	31.5	301	37.1
noisy / all-noise	0	0.0	1	0.1
partition total	1631	100	811	100
<i>merged</i> (cross-cutting)	8	0.5	0	0.0

Three things follow. First, **splitting, not merging, is the failure mode**: 31 % to 37 % of cars split, while merging is at most 0.5 %. The pipeline almost never fuses two cars into one detection; it repeatedly breaks one car into several. Second, **the split is worst where cars are largest and nearest**: the split rate falls monotonically with range — 66 % at 0 m to 10 m, 40 % at 10 m to 20 m, 19 % at 20 m to 30 m, and 4 % at 30 m to 50 m. Large, close cars return many points, and HDBSCAN finds internal density variations within that dense return — the roof line against the body, the near side against the far side — and cuts the car along them. This is the mechanism behind the counter-intuitive near-range recall dip reported in Section 4.3.4: the nearest bin has lower recall than the 10 m to 20 m bin despite carrying far more points, because a close car dissolves

into fragments that no longer match the whole instance. Third, **the cleanly-clustered fraction tracks recall, and splitting usually loses the car**. The fraction of cars that form one dominant cluster (68.5 % on sequence 00, 62.8 % on sequence 08) sits just below the recall the pipeline achieves on the same configuration (0.739 and 0.699 respectively). Two things follow. The pipeline recovers essentially every cleanly-clustered car, so that fraction is close to a floor on recall — not a ceiling; recall exceeds it only by the few split cars whose dominant fragment happens to survive matching and filtering. And because recall (0.699) barely exceeds the clean fraction (62.8 %) while 37.1 % of cars split, the great majority of split cars are not recovered: a large dominant point-fraction (median 0.77–0.92) does not translate into a match, because matching depends on spatial overlap and on the fragment clearing the size filters, not on point count. Splitting is therefore the mechanism by which cars are lost, which is exactly why it is the recall bottleneck. This also predicts the resolution fix of Section 4.7.4: coarsening the voxel converts split cars into clean ones, which should lift recall by shrinking the reliance on recovering splits — and it does, narrowing the sequence-08 gap between the clean fraction and recall from 7.1 points (62.8 % versus 0.699) to 3.1 points (69.9 % versus 0.730).

Figure 4.7 shows the mechanism directly on three sequence 08 frames. Missed cars (orange) appear as broken rows of points close to the ego vehicle, each row too small on its own to survive the size filters, while cleaner mid- and far-range cars (green) are detected as single clusters.

4.7.3 Limits of the Repair Strategies

Given that splitting caps recall, the obvious responses fall into two families: change how clusters *form* (a larger minimum cluster size, a different clusterer, distance-adaptive parameters, or pre-clustering point accumulation) or reassemble fragments after HDBSCAN has split them. We tried six in total; Table 4.18 collects the four with a reproducible quantitative outcome, and two further strategies (temporal accumulation and threshold lowering) are discussed in prose below. All were evaluated against the same pre-promotion sequence-00 baseline (full pipeline, classifier and track filter on; $F_1 = 0.844$), and the load-bearing ones were re-checked on the held-out sequence 08.

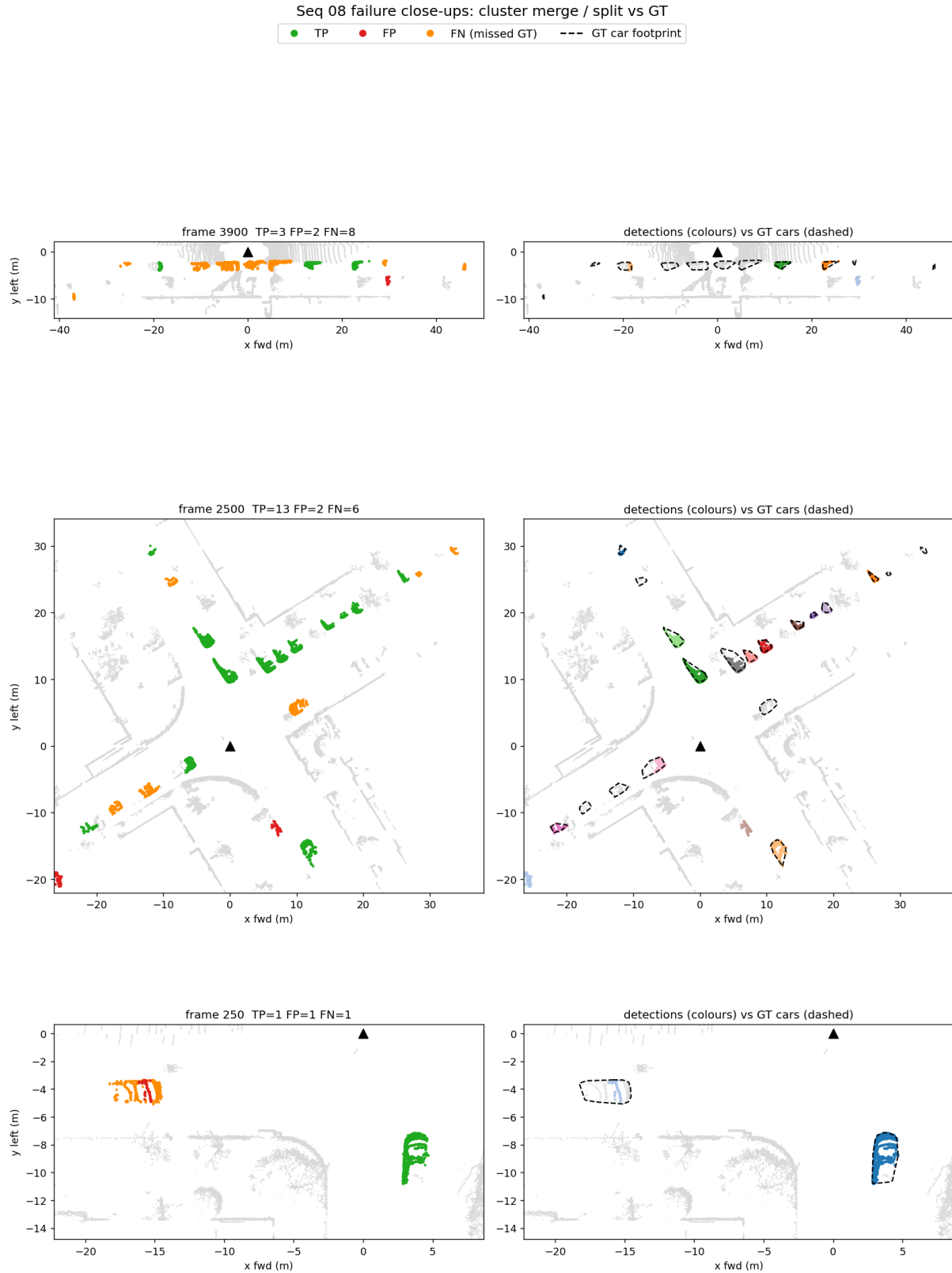


Figure 4.7: Failure close-ups on sequence 08 (true positives green, false positives red, missed ground-truth cars orange; dashed outlines are ground-truth car footprints). Near-ego missed cars are fragmented into several small point groups rather than one cluster, matching the split mechanism of Table 4.17. Although the panel title mentions both merge and split, merging is measured at $\leq 0.5\%$ (Table 4.17), so the misses shown are fragmentation, not fusion of neighbouring cars.

Table 4.18: Recall-repair strategies against the pre-promotion baseline (full pipeline, sequence 00, 100 frames; held-out sequence-08 F_1 where measured). Every strategy either loses F_1 outright or fails to hold its sequence-00 gain on the held-out sequence. This is the *pre-promotion* repair search — it predates the coarse-voxel configuration of Section 4.7.4, which is why the baseline here is $F_1 = 0.844$, not the promoted headline. Sources in the “Src” column are finding numbers. Two further strategies — temporal point accumulation and lowering the size thresholds — are discussed in the text but omitted here: temporal accumulation is single-sourced to a reverted feature in the session log (not reproducible under the research freeze), and threshold lowering produced no change in true positives; both belong in prose rather than the main results table.

Strategy	Seq 00 P	Seq 00 R	Seq 00 F_1	Seq 08 F_1
Baseline (HDBSCAN, MCS 10)	0.984	0.739	0.844	0.829
BEV clustering (res 0.30 m)	0.993	0.640	0.779	—
Higher MCS 20	0.976	0.755	0.852	0.801
Fragment merge (1.5 m, 30 pt)	0.931	0.755	0.834	0.815
Adaptive HDBSCAN (30/15/10)	0.935	0.748	0.831	—

Each strategy fails for a structural reason, and the reasons are consistent with the diagnosis of Section 4.7.2:

- **Higher minimum cluster size (MCS 20)** is the most instructive. It is the only variant to improve F_1 on sequence 00 ($0.844 \rightarrow 0.852$), by discouraging the small splinter clusters. But it does not generalise: on the held-out sequence 08 it drops to F_1 0.801, below the 0.829 baseline, because a larger minimum also erases genuinely sparse distant cars. The sequence-00 gain was over-fitting.
- **Post-clustering fragment merge** (join clusters within 1.5 m) raises sequence-00 recall to 0.755 but loses precision ($0.984 \rightarrow 0.931$), because the same proximity rule also fuses nearby non-car structure — walls, poles — into oversized clusters. On held-out data the precision loss cancels the recall gain (F_1 0.815 < 0.829).
- **Distance-adaptive HDBSCAN** (a larger minimum cluster size near the ego, smaller far away) introduces boundary artefacts at the ring transitions and is worse than the global setting.
- **Bird’s-eye-view clustering**, a cheaper connected-component alternative, collapses recall to 0.640: projecting to two dimensions merges objects that overlap in the ground plane but are separated in height, and morphological cleanup destroys sparse distant clusters.
- **Temporal aggregation** — accumulating object points over three consecutive frames before clustering, to densify sparse cars — collapses the pipeline (F_1 0.073, recall 0.039).² Accumulating points does not densify the existing clusters; it reshapes the clustering landscape entirely (about 134 000 accumulated points yield 645+ clusters versus ~240 single-frame, of which fewer pass the geometric filters, 17 versus 34).

²Source: docs/session_history.md (2026-06 exploratory session). The temporal_window feature was implemented and then fully reverted, so this result is on the pre-promotion baseline and was not re-run under the research freeze; it is reported here in prose, not in Table 4.18, for that reason, and its value is the mechanism below rather than the exact figure.

- **Lowering the cluster-size thresholds** to admit sparser groups produced no change in true positives at all: the binding constraint is cluster quality, not the count threshold, so relaxing the threshold merely admits more noise (`docs/session_history.md`).

The failure modes are strategy-specific — over-fitting, precision loss, ring artefacts, projection ambiguity, a reshaped clustering landscape — but they share a limit. The two strategies that reassemble after the split (fragment merge, and to a lesser extent the proximity-based part of the adaptive scheme) run into the fact that density-based clustering has no notion of objectness: once a dense car is partitioned along an internal density gap, no proximity- or size-based rule can reliably tell “rejoin these two fragments of one car” apart from “do not join these two adjacent cars.” The strategies that instead change how clusters *form* — a larger minimum size, a different clusterer, per-ring parameters, pre-clustering accumulation — avoid the reassembly trap but hit the same wall from the other side: any single global density criterion that is loose enough to keep a split car whole is also loose enough to fuse a close pair, so a sequence-00 gain becomes a held-out loss. The one lever that helped, changing the clustering *resolution* (Section 4.7.4), is also a formation change; it works not by escaping this limit but by happening to sit at a resolution that closes intra-car gaps before it starts closing inter-car ones. This is why the earlier project conclusion — that recall was fixed at a hard ceiling of about 0.74 — was reasonable given only the evidence in Table 4.18, and also, as the next section shows, wrong: it read a limit on these repairs as a limit on the approach.

4.7.4 Clustering Resolution and the Corrected Ceiling

The intervention that helped was not a repair at all but a change to the clustering *resolution*: re-voxelising the object cloud to a 0.10 m grid before running HDBSCAN, rather than clustering the full-resolution 0.05 m cloud (adopted in the runtime optimisation of Section 4.4). A coarser grid closes the small intra-car density gaps that HDBSCAN was cutting along, so fewer splits form in the first place. It is a formation change like the failed ones in Section 4.7.3, but it lands on the favourable side of the trade described there — coarse enough to close intra-car gaps, still fine enough not to fuse neighbouring cars.

Table 4.19: Effect of coarsening the clustering voxel from 0.05 m to 0.10 m on the split rate (sequence 08, 100 frames, 810 ground-truth cars). More cars cluster cleanly and none newly merge — the coarsening removes splits without fusing neighbours.

	Full-res (0.05 m)	Coarse (0.10 m)
clustered “ok”	504 (62.2%)	566 (69.9%)
split	305 (37.7%)	242 (29.9%)
merged	0 (0.0%)	0 (0.0%)
mean clusters per split car	3.3	2.7

Table 4.19 shows the split rate falling by 7.8 points (37.7% $\rightarrow$ 29.9%) with zero new merges.³ At the pipeline level, on the full sequence 08, this lifts recall from 0.699 to 0.730

³The full-resolution counts here (810 cars, split 37.7%) and those in Table 4.17 (811 cars, split 37.1%) are separate runs of the same analysis on the same pre-promotion configuration; they differ by at most one car and 0.6 percentage points, and the percentages are unaffected.

(+1655 true positives) with precision essentially unchanged ($0.903 \rightarrow 0.905$) and F_1 from 0.788 to 0.808. The same coarsening also reduces clustering runtime, which is why it was adopted; its detection benefit is the subject here and its runtime co-benefit is reported in Section 4.4.

This result forces a correction to the earlier ceiling claim, which we state plainly rather than bury. The “ ~ 0.74 hard limit” was *partly a resolution artefact*, not a fundamental property of the approach: the strategies of Section 4.7.3 genuinely do fail, but none of them tried this particular resolution setting, and changing it recovered part of the loss. The standing recommendation is therefore `cluster_voxel_size = 0.10m` in production, not “accept 0.74.”

A smaller structural limit does remain, and it is architectural. Coarsening the voxel further would eventually start merging adjacent cars, trading the split problem for a merge problem; and the clustering-method benchmark of Section 4.6 shows that no fixed-radius density clusterer, at any tested radius, reaches even HDBSCAN’s recall. The ceiling that remains is the one imposed by density-based clustering having no model of what a car is: it groups points by proximity and density alone, so it cannot be pushed to the recall of a learned object detector by tuning. Reaching that level would require replacing the clustering stage with a learned detector, which is outside the scope of this thesis (Chapter 5). The value of this analysis is to have located the ceiling precisely — it is in the clustering stage, it is splitting rather than classification or merging, and it is lower than first claimed — and to have mapped which repairs cannot move it and which one can.

4.7.5 Recall Analysis: Summary

The recall shortfall is clustering-bound, not classifier-bound: the classifier and track filter sit on top of a recall ceiling that geometric clustering already imposes (Section 4.7.1). The dominant mechanism is HDBSCAN splitting large, close cars into fragments — 31 % to 37 % of cars, worst at short range — while merging is negligible, and the cleanly-clustered fraction sits just below the achieved recall, so most split cars are lost (Section 4.7.2). A search of six repair strategies — higher minimum cluster size, fragment merging, adaptive and BEV clustering, temporal accumulation, and threshold lowering — all fail to generalise, for a shared reason: density-based clustering has no notion of objectness, so any criterion loose enough to keep a split car whole also fuses adjacent cars, and reassembling fragments after the split faces the same ambiguity (Section 4.7.3). The one effective lever was clustering resolution: a coarser pre-clustering voxel prevents splits from forming, lifting recall $0.699 \rightarrow 0.730$ and correcting the earlier overstated “hard ceiling” (Section 4.7.4). The residual limit is architectural — density clustering has no notion of objectness — and is carried to the limitations and future-work discussion of Chapter 5.

With the detection front-end characterised, we now put the donor metric through its validation battery and use it to carry the completion evaluation. The donor-frame metric it uses to score completion was defined in Section 3.4.3. Because that metric carries the completion claims, we first show it behaves — passing an explicit validation battery — and then report what it, together with the amodal-box evaluation, reveals about the completion output. The metric is defined on static cars; moving cars are treated separately in Section 4.5.4.

Chapter 5

Conclusion and Future Work

The preceding chapters reported results; this one interprets them and closes the thesis. It first draws the conclusions, answering the two research questions and restating the contributions (Section 5.1); then gathers what the evidence supports and how far each claim reaches, together with every limitation the work carries, into a single section (Section 5.2) — the validity of the evaluation, the scope of the completion claims, the ground-truth assumptions, and the architectural and operational limits — and finally turns the most consequential boundaries into future work (Section 5.3). Every weakness named here is measured elsewhere in the document and cross-referenced to the finding that quantifies it: a limitation the thesis names, quantifies, and bounds is defended; one a committee surfaces first is not.

5.1 Conclusion

Chapter 1 posed two research questions, both about measuring completion on real LiDAR; each is answered in full in Section 4.5.1 and restated here in the form asked, with its reach discussed in Section 5.2.1.

RQ1 — how to evaluate completion on real LiDAR. Reference-based Chamfer distance against accumulated pseudo-ground-truth is invalid, because the accumulated reference is occluded on the same side as the input and so rewards under-completion; the thesis replaces it with the donor-frame occluded-side coverage metric, which scores recovery of surface seen only in other frames of the same static car and passes a four-item validation gate.

RQ2 — whether completion recovers unseen surface, and whether the benefit holds out. Yes, within a named scope. On sequence 08 the donor metric shows PCN recovers genuinely occluded surface (coverage at 0.1 m $0.000 \rightarrow 0.304$ against the mirror baseline 0.043) and improves the amodal box ($0.725 \rightarrow 0.771$); under a pre-registered test on held-out sequence 00 the benefit PARTIALLY HOLDS — both metrics replicate, but the long band was untestable and the compact guard fails. Moving cars are a plausibility check only.

Diagnostic findings. Two findings emerged along the way (Section 4.7, Chapter 4): recall is clustering-bound, not classifier-bound — HDBSCAN splits 31 % to 37 % of cars

and a chain of repair strategies fails to generalise — and synthetic pretraining is redundant once real clusters are plentiful, the sim-to-real gap being total and symmetric.

Contributions. The thesis makes a single scientific contribution, methodological rather than architectural: **a validated metric for completion quality on real LiDAR**, through which learned completion is shown to recover genuinely occluded surface and improve amodal boxes on static cars, partially replicated on a held-out sequence. Two diagnostic findings sit alongside it — the clustering-bound recall account and the redundancy of synthetic pretraining — with engineering contributions (the modular pipeline and its runtime characterisation, the amodal box builder, the reproducibility infrastructure) and documented negative-result lines.

5.2 Limitations and Discussion

5.2.1 Established Results and Their Reach

Two threads carry the strongest evidence, both belonging to the metric contribution. The first is methodological: the donor-frame occluded-side coverage metric (Section 4.5.1). We are not aware of a prior metric that scores a real LiDAR car’s completion against donor frames of the same static instance — the reference-free measures reviewed in Section 2.1.4 all score against the observed input, a synthetic prior, or the model’s own outputs, and the closest prior construction (Ren et al. [27]) scores the accumulated cloud as a whole, not the never-observed surface. It also carries a measured negative result about metrics: the obvious alternative — Chamfer distance against accumulated pseudo-ground-truth — is invalid, because accumulated LiDAR is itself one-sided and rewards under-completion (the raw partial scored the lowest Chamfer distance on every real example), as do the field’s standard input-referenced UCD/UHD (Section 4.5.2). The second thread is empirical and rests on that metric: completion adds genuinely unseen surface (median occluded-side coverage within 0.1 m of 0.304 on sequence 08, against 0.043 for the symmetry mirror and 0.000 for the raw partial) and improves the amodal box (median BEV IoU $0.725 \rightarrow 0.771$, gains concentrated where the sensor saw least); both survive a pre-registered held-out replication with named caveats (PARTIALLY HOLDS; Section 5.2.2).

The detection pipeline is a strong engineering result rather than a claim of state-of-the-art accuracy: the supervised networks positioned in Section 2.1.5 reach higher accuracy at the dense annotation cost this pipeline avoids. It reaches precision 0.905, recall 0.730, F_1 0.808 on the full sequence 08 (mean IoU of matched detections 0.912; recall denominator: car instances with at least 10 points surviving preprocessing, Section 4.2), with an interpretable, near-annotation-free clustering front end on modest hardware. Its recall shortfall is root-caused, not mysterious — HDBSCAN splitting large, close cars into fragments (Section 4.7) — and that diagnosis, with the finding that synthetic pretraining is redundant at scale, are the two secondary results the investigation produced.

These reaches vary. The detection operating point is the broadest, cross-validated across all eleven labelled sequences (Section 4.3.3). Two completion results transfer by argument rather than by sample — the invalidity of accumulated-Chamfer follows from the one-sidedness of accumulated LiDAR, and the donor metric is dataset-agnostic wherever static cars are seen from multiple frames — whereas the recall diagnosis is an inductive result from two sequences, mechanistic but not a proven invariant. The tuned constants (the length prior, the estimate quantile, the geometric thresholds) are

sequence-derived and should be re-checked on a new sensor, and the completion accuracy claims are scoped to static, gate-passed cars in the compact and normal bands. For practice, three readings follow: the interpretable, near-annotation-free detector is a viable choice where labelling budget or inspectability outweighs peak recall, provided the domain is dense-scene; the donor metric is a reusable instrument for any real-LiDAR completion where static objects recur across frames; and because input quality, not model choice, governs whether completion is usable, effort is better spent on segmentation than on swapping architectures. These reaches are bounded by the limitations set out next.

5.2.2 Validity of the Evaluation

Detection is cross-validated, and the selection exposure is precision-only. SemanticKITTI labels only sequences 0 to 10, so no labelled sequence can be reserved as a conventional held-out detection test set, and the final classifier was historically selected on sequence 08; leave-one-sequence-out cross-validation (Section 4.3.3) bounds the exposure, the fold-08 classifier matching its recall (0.737 versus 0.730) and losing only about two precision points (0.881 versus 0.905), so the selection touched precision, not the clustering-imposed recall ceiling (0.775) that accounts for most missed cars. Completion is held out more strongly still — its claims are re-tested on sequence 00 under a pre-registration and the model never saw a real sequence — and that replication returned PARTIALLY HOLDS: both primary metrics were decisive wins (BEV IoU $0.739 \rightarrow 0.766$, $p = 1.6 \times 10^{-3}$; donor coverage $0.000 \rightarrow 0.413$), but the long band was empty (no car ≥ 4.6 m survived the static-car construction), leaving long-car behaviour untestable off the tuning sequence, and the compact-band guard fails there as on sequence 08. The completion value is thus replicated once, not across a range of sequences.

5.2.3 Scope of the Completion Claims

The quantitative completion claims are bounded in what they can score. Both metrics require a static car by construction, so moving cars cannot be scored by either and are reported only as a plausibility check, where they reach plausible boxes at essentially the static rate (57.9% versus 53.7%) — a parity result, not an accuracy claim, and a conservative one, since the axis-aligned flag penalises the non-axis-aligned cars movers over-represent. The metric also scores only clean, gate-passed clusters, and whether a car reaches completion as one is decided upstream by the clustering that caps recall: the ablations (Section 4.6.2) show the L-shape input gate sharply raises plausible-completion rates on every sequence whereas swapping PCN for PoinTr changes them not at all, so widening completion’s reach is a segmentation problem, not a completion-architecture one.

Within that scope, three residuals bound accuracy. The per-car length estimate falls back to the 4.14 m population prior on tracks with fewer than five gate-passed frames — 23.0% of sequence-08 tracks — reintroducing compact overshoot there; the cost sits inside the headline gain (median BEV IoU $0.725 \rightarrow 0.771$, measured on the same tracks) and falls on the sparsest, most distant cars where the box matters least. Even with an unbiased estimate the completed cloud stays slightly short at the far end by an amount that scales with car length, and a pre-registered fix removed this on normal and long cars but over-extended compacts, so the compensation is irreducibly length-dependent and the far end remains the weakest-covered region. Both residuals compound the model’s synthetic-only training — the source of the evaluation’s independence but also the synthetic-to-real gap

(Section 2.1.4) behind the residual flatness of real completions — which shows most sharply on compact cars, where completions spray points outside the box at roughly 100 times the mirrored baseline under the fixed prior and about 16 times under the per-car estimate; no configuration clears the compact bar, so we report the ratios rather than tune the guard, a band-localised failure the original pooled-median guard was blind to.

5.2.4 Ground-Truth and Dataset Assumptions

Three assumptions underlie the ground truth. The amodal box is constructed by accumulating a static car’s frames, since no external amodal reference exists (KITTI raw tracklets for the sequence-08 drive, `2011_09_30_drive_0028`, do not exist on the official archive); it rests on the paired design comparing raw against completed on the same box, viewpoint- and motion-coverage guards, and a sane dimension distribution, so the claim is robustness to plausible reference error, not an error-free reference. Recall is reported against eligible cars — those with at least 10 points surviving preprocessing — which excludes 25.5 % of annotated cars too sparse or distant to reach the front end, so recall against all annotated cars is approximately 0.54 against the 0.730 eligible figure (both reported; the two “ ≥ 10 -point” rules are distinct). And all counts, matches, and the amodal construction assume SemanticKITTI’s labels are correct; we did not audit them, and mislabelled cars would propagate into both the detection metrics and the ground truth.

5.2.5 Architectural and Operational Limits

The remaining limits are architectural or operational. The recall ceiling is architectural, not a tuning failure (Section 4.7): coarsening the cluster voxel to 0.10 m recovered part of the loss (0.699 $\rightarrow$ 0.730) but coarsening further fuses adjacent cars, and no fixed-radius density clusterer reaches HDBSCAN’s recall at any tested radius, so the ceiling (0.775 before any learned stage, splitting 31 % to 37 % of cars) is the measured price of the interpretable, near-annotation-free design the thesis exists to evaluate, and lifting it means a learned detector — a different system. The operational domain is dense scenes: recall falls to 0.336 on the sparse highway sequence 01 against 0.761 on dense-urban sequence 00, mostly because highway cars are distant and recall collapses with range (0.196 beyond 40 m) and partly because the front end was tuned on dense data, though the failure is a conservative miss, not a false alarm (sequence 01 precision 0.951). The system is offline by design, running at about 623.5 ms per frame (Section 4.4) with the ≤ 400 ms target unreachable through the authorised tiers, and real-time operation is out of scope. Finally, the centroid tracker feeds the length estimate and drives the 23 % fallback but is never scored as a tracker (no MOTA or IDF1), so its contribution to that fallback is unmeasured.

5.3 Future Work

The limitations above named the boundaries of the present results. Three of them are the natural next steps, each already scoped by a finding rather than left open.

A real-time variant. The pipeline runs offline at roughly 623.5 ms per scan (Section 4.4), dominated by the classical CPU stages. The direct speed path is an implementation one: the residual ~ 185 ms attributed to clustering is the cost of the *Python* HDBSCAN, not the HDBSCAN *algorithm*, so a native or GPU implementation would attack that floor

without changing what is computed. Swapping in a faster fixed-radius clusterer is not a free substitution — the benchmark shows none reaches HDBSCAN’s recall at any tested radius — so a real-time variant must preserve the HDBSCAN algorithm via a faster implementation, or replace the front end with a learned detector, rather than trade measured recall for speed.

Real-data fine-tuning for completion. The completion model was trained on synthetic KITTI-like partials only, which is at once the source of the donor metric’s validity and a ceiling on real-data crispness: on the sparsest, most one-sided inputs the completions stay flat, which traces to the remaining synthetic-to-real domain gap. Closing it would call for real-data fine-tuning, which is itself a documented negative-result line in this project; it is a contingency direction, to be attempted only with the leakage controls that make real-data supervision safe.

Long-band amodal ground truth on a second held-out sequence. The held-out replication (Section 4.5.5) partially held in part because the long length band was empty on sequence 00 — no car of at least 4.6 m survived the well-observed-static-car construction — so the per-car length estimate’s behaviour on long cars could not be tested off the tuning sequence. Building amodal ground truth for a third sequence with long cars present would close that single coverage gap and upgrade the long-band claim from a sequence-08 result to a replicated one. It is deferred here as a bounded data-construction task, not a change to the method.

None of the thesis’s results rests on an additional point of metric improvement: the methodological contribution is a validated real-data completion metric, and each empirical claim — that learned completion adds genuinely unseen surface and improves amodal boxes on static cars, that recall is clustering-bound, that synthetic pretraining is redundant at scale — is defended by explanation and, where the design allowed it, by a pre-registered test whose outcome was reportable either way. That is the standard the work set for itself, and the standard on which it asks to be judged.

Bibliography

- [1] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [2] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, “Semantickitti: A dataset for semantic scene understanding of lidar sequences,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.
- [3] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.
- [4] C. R. Qi, L. Yi, H. Su, and L. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/d8bf84be3800d12f74d8b05e9b89836f-Paper.pdf
- [5] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham, “Randla-net: Efficient semantic segmentation of large-scale point clouds,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [6] B. Wu, A. Wan, X. Yue, and K. Keutzer, “Squeezeseg: Convolutional neural nets with recurrent crf for real-time road-object segmentation from 3d lidar point cloud,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE Press, 2018, p. 1887–1893. [Online]. Available: <https://doi.org/10.1109/ICRA.2018.8462926>
- [7] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, “Rangenet++: Fast and accurate lidar semantic segmentation,” in *2019 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2019, pp. 4213–4220.
- [8] X. Zhu, H. Zhou, T. Wang, F. Hong, Y. Ma, W. Li, H. Li, and D. Lin, “Cylindrical and asymmetrical 3d convolution networks for lidar segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [9] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “Pointpillars: Fast encoders for object detection from point clouds,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

- [10] T. Yin, X. Zhou, and P. Krähenbühl, “Center-based 3d object detection and tracking,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [11] Z. Zhou, Y. Zhang, and H. Foroosh, “Panoptic-polarnet: Proposal-free lidar point cloud panoptic segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [12] F. Hong, H. Zhou, X. Zhu, H. Li, and Z. Liu, “Lidar-based panoptic segmentation via dynamic shifting network,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [13] N.-Y. Lim and T.-H. Park, “Long-range lidar vehicle detection through clustering and classification for autonomous racing,” *IEEE Access*, vol. 13, 2025.
- [14] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, “Pcn: Point completion network,” 2019. [Online]. Available: <https://arxiv.org/abs/1808.00671>
- [15] J. Behley, A. Milioto, and C. Stachniss, “A benchmark for lidar-based panoptic segmentation based on kitti,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021.
- [16] S. Gasperini, M.-A. N. Mahani, A. Marcos-Ramiro, N. Navab, and F. Tombari, “Panoster: End-to-end panoptic segmentation of lidar point clouds,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3216–3223, 2021.
- [17] K. Sirohi, R. Mohan, D. Büscher, W. Burgard, and A. Valada, “Efficientlps: Efficient lidar panoptic segmentation,” *IEEE Transactions on Robotics*, vol. 38, no. 3, 2022.
- [18] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 1996, pp. 226–231.
- [19] R. J. G. B. Campello, D. Moulavi, and J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD)*. Springer, 2013, pp. 160–172.
- [20] L. McInnes, J. Healy, and S. Astels, “hdbscan: Hierarchical density based clustering,” *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.
- [21] X. Yu, Y. Rao, Z. Wang, Z. Liu, J. Lu, and J. Zhou, “Pointtr: Diverse point cloud completion with geometry-aware transformers,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 12 498–12 507.
- [22] P. Xiang, X. Wen, Y.-S. Liu, Y.-P. Cao, P. Wan, W. Zheng, and Z. Han, “Snowflakenet: Point cloud completion by snowflake point deconvolution with skip-transformer,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 5499–5509.
- [23] H. Zhou, Y. Cao, W. Chu, J. Zhu, T. Lu, Y. Tai, and C. Wang, “Seedformer: Patch seeds based point cloud completion with upsample transformer,” in *European conference on computer vision*. Springer, 2022, pp. 416–432.

- [24] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, “Shapenet: An information-rich 3d model repository,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03012>
- [25] X. Chen, B. Chen, and N. J. Mitra, “Unpaired point cloud completion on real scans using adversarial training,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [26] R. Cui, S. Qiu, S. Anwar, J. Liu, C. Xing, J. Zhang, and N. Barnes, “P2C: Self-supervised point cloud completion from single partial clouds,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [27] Y. Ren, P. Cong, X. Zhu, and Y. Ma, “Self-supervised point cloud completion on real traffic scenes via scene-concerned bottom-up mechanism,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.10569>
- [28] X. Weng, J. Wang, D. Held, and K. Kitani, “3d multi-object tracking: A baseline and new evaluation metrics,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [29] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [30] H. Fan, H. Su, and L. J. Guibas, “A point set generation network for 3D object reconstruction from a single image,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.